

*Under the guidance  
of*  
**PROF. S. P. BHOLANE**



**Sinhgad Institiutes**

**DEPARTMENT OF COMPUTER ENGINEERING**  
**SINHGAD COLLEGE OF ENGINEERING, PUNE-41**  
*Accredited by NAAC with A+ Grade*  
**2025-26**

A PRELIMINARY PROJECT REPORT  
ON  
**JURISAI: SMART LEGAL ADVISOR  
AND DOCUMENT ASSISTANT**

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE  
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS  
FOR THE AWARD OF THE DEGREE

**BACHELOR OF ENGINEERING**

*In*

**COMPUTER ENGINEERING**

*of*

**SAVITRIBAI PHULE PUNE UNIVERSITY**

*By*

**DESAI SHWETA PUNDALIK**  
**DEOKAR SAMRUDDHI SATISH**  
**DESHMUKH RIYA RAJARAM**  
**DHONDGE MANAS SATISH**

**PRN NO: 72230913K**  
**PRN NO: 72230910E**  
**PRN NO: 72230923G**  
**PRN NO: 72230953J**

**Roll No: 405A015**  
**Roll No: 405A017**  
**Roll No: 405A018**  
**Roll No: 405A019**

Date:

## **CERTIFICATE**

This is to certify that the preliminary project report entitled

**“JURISAI: SMART LEGAL ADVISOR AND DOCUMENT ASSISTANT”**

Submitted by

**DESAI SHWETA PUNDALIK**

**PRN NO:72230913K Roll No: 405A015**

**DEOKAR SAMRUDDHI SATISH**

**PRN NO:72230910E Roll No: 405A017**

**DESHMUKH RIYA RAJARAM**

**PRN NO:72230923G Roll No: 405A018**

**DHONDGE MANAS SATISH**

**PRN NO:72230953J Roll No: 405A019**

is a bonafide work carried out by her under the supervision of Prof. S. P. Bholane and it is approved for the partial fulfillment of the requirements of Savitribai Phule Pune University, Pune for the award of the degree of Bachelor of Engineering (Computer Engineering) during the year 2025-26.

Prof. S. P. Bholane  
Guide  
Department of Computer Engineering

Dr. S. R. Ganorkar  
Head  
Department of Computer Engineering

Dr. S. D. Lokhande  
Principal  
Sinhgad College of Engineering

## Acknowledgement

I take this opportunity to express our profound sense of gratitude and respect to all those who directly or indirectly helped me through the duration of this work. It gives me great pleasure to present a preliminary project report on **JURISAI: SMART LEGAL ADVISOR AND DOCUMENT ASSISTANT**. I am extremely obliged to our major project guide **Prof. S. P. Bholane** in the Department of Computer Engineering for helping and providing me with proper guidance. Timely suggestions made it possible for me to complete this seminar successfully.

I will fail in my duty if I don't acknowledge a great sense of gratitude to the head of the computer engineering department **Dr. S. R. Ganorkar** and the entire staff members of our department for their cooperation.

DESAI SHWETA PUNDALIK PRN NO:72230913K Roll No: 405A015

DEOKAR SAMRUDDHI SATISH PRN NO:72230910E Roll No: 405A017

DESHMUKH RIYA RAJARAM PRN NO:72230923G Roll No: 405A018

DHONDGE MANAS SATISH PRN NO:72230953J Roll No: 405A019

## **Abstract**

Access to reliable legal information and standardized documentation remains a significant challenge for individuals and small businesses, largely due to language barriers, lack of legal expertise, and the inherent complexity of legal texts. JurisAl: Smart Legal Advisor and Document Assistant address these challenges by leveraging artificial intelligence to provide accessible, transparent, and efficient legal support.

The system integrates four core modules:

- (i) a constitution-based legal advisor chatbot that delivers accurate, source-cited guidance;
- (ii) multilingual proprietary document generation using standardized templates to facilitate the creation of legally valid contracts and forms;
- (iii) a document summarization engine that condenses lengthy legal texts into concise, easy-to-understand formats;
- (iv) an agreement clause verification tool that detects ambiguous terms, potential loopholes, and high-risk provisions in contracts.

By combining natural language processing, legal knowledge representation, and multilingual support, JurisAl aims to enhance legal literacy, reduce dependency on costly legal consultations, and empower users with the ability to make informed decisions. This project contributes toward democratizing access to legal resources, fostering transparency, and promoting trust in legal documentation processes.

## **List of Figures**

|                                       |    |
|---------------------------------------|----|
| 2.4.3 Gantt Chart                     | 17 |
| 3.5 Architecture Diagram              | 25 |
| 3.6.1 Use Case Diagram                | 26 |
| 3.6.2 Sequence Diagram                | 26 |
| 3.6.3 Activity Diagram                | 27 |
| 4.1 Legal Advisor                     | 36 |
| 4.2 Document Summarisation            | 37 |
| 4.3 Multilingual Document Generation  | 38 |
| 4.4 Agreement Clause Verification     | 39 |
| 6.1 Account Creation                  | 42 |
| 6.2 Client View                       | 43 |
| 6.3 Lawyer View                       | 43 |
| 6.4 Legal Advisor                     | 44 |
| 6.5 Document Summarisation            | 44 |
| 6.6 Multilingual Document Generation  | 45 |
| 6.7 Agreement Clause Verification     | 46 |
| 6.8 Response Time vs Query Complexity | 47 |
| 6.9 Distribution of User Queries      | 47 |
| 6.10 Document Size vs Processing Time | 48 |
| 6.11 Module Performance Comparison    | 48 |

## **List of Tables**

|       |                                 |    |
|-------|---------------------------------|----|
| 2.2.3 | Non-Functional Requirements     | 11 |
| 2.2.4 | Deployment Environment          | 11 |
| 2.2.5 | External Interface Requirements | 12 |
| 2.4.2 | Project Timeline Overview       | 15 |
| 3.2   | IDEA Matrix                     | 18 |
| 3.4.4 | Computational Complexity        | 23 |
| 3.4.6 | Overall Feasibility Summary     | 23 |
| 5.1   | Unit Testing                    | 40 |
| 5.2   | Integration Testing             | 41 |
| 5.3   | Functional Requirements         | 41 |
| 5.4   | Non-Functional Requirements     | 41 |

## Acronyms

|     |                                     |
|-----|-------------------------------------|
| SRS | Software Requirements Specification |
| UML | Unified Modelling Language          |
| AI  | Artificial Intelligence             |
| LA  | Legal Advisor                       |
| MDG | Multilingual Document Generation    |
| DS  | Document Summarisation              |
| ACV | Automated Clause Verification       |
| INT | Integration Testing                 |
| FA  | Functional Acceptance               |
| NFA | Non-Functional Acceptance           |

## TABLE OF CONTENTS

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Title page</b>                                   |           |
| <b>Certificate page</b>                             |           |
| <b>Acknowledgement</b>                              | (i)       |
| <b>Abstract</b>                                     | (ii)      |
| <b>List of Figures</b>                              | (iii)     |
| <b>List of Tables</b>                               | (iv)      |
| <b>Acronyms</b>                                     | (v)       |
| <b>1 INTRODUCTION</b>                               | <b>1</b>  |
| 1.1 Background and basics                           | 2         |
| 1.2 Literature Survey                               | 4         |
| 1.3 Project Undertaken                              | 7         |
| 1.3.1 Problem definition                            | 7         |
| 1.3.2 Scope Statement                               | 7         |
| <b>2 PROJECT PLANNING AND MANAGEMENT</b>            | <b>8</b>  |
| 2.1 Introduction                                    | 8         |
| 2.2 System Requirement Specification (SRS)          | 8         |
| 2.3 Project Process Modeling                        | 12        |
| 2.4 Project Scheduling                              | 15        |
| <b>3 ANALYSIS &amp; DESIGN</b>                      | <b>18</b> |
| 3.1 Introduction                                    | 18        |
| 3.2 IDEA matrix                                     | 18        |
| 3.3 Mathematical Model                              | 20        |
| 3.4 Feasibility Analysis (NP Completeness Analysis) | 21        |
| 3.5 Architecture Diagram                            | 25        |
| 3.6 UML diagrams                                    | 26        |
| <b>4 IMPLEMENTATION &amp; CODING</b>                | <b>28</b> |
| 4.1 Introduction                                    | 28        |
| 4.2 Operational Details                             | 28        |
| 4.3 Screenshots                                     | 36        |
| <b>5 TESTING</b>                                    | <b>40</b> |
| 5.1 Introduction                                    | 40        |



|            |                                         |    |
|------------|-----------------------------------------|----|
| <b>5.2</b> | Unit Testing                            | 40 |
| <b>5.3</b> | Integration Testing                     | 41 |
| <b>5.4</b> | Acceptance Testing                      | 41 |
| <b>6</b>   | <b>RESULTS &amp; DISCUSSIONS</b>        | 42 |
| <b>6.1</b> | Main GUI Snapshots                      | 42 |
| <b>6.2</b> | Table of the results/findings or graphs | 47 |
| <b>6.3</b> | Discussions                             | 49 |
| <b>7</b>   | <b>CONCLUSION</b>                       | 50 |
| <b>8</b>   | <b>FUTURE WORK</b>                      | 51 |
|            | <b>References</b>                       | 52 |
|            | <b>Annexure</b>                         | 54 |

---

# Chapter 1

## Introduction

In today's fast-paced digital era, the legal domain remains one of the most complex and inaccessible areas for the general public. Individuals and small businesses often face significant challenges in understanding legal documents, drafting contracts, and ensuring compliance with constitutional or regulatory frameworks. These challenges arise due to multiple factors such as lack of legal awareness, linguistic barriers, and the intricate nature of legal terminology. Consequently, many people remain dependent on expensive legal consultations or unreliable online resources, leading to misunderstandings and potential disputes.

To address these issues, **JurisAI: Smart Legal Advisor and Document Assistant** aims to revolutionize the accessibility and transparency of legal services through the power of Artificial Intelligence. The system leverages modern AI and Natural Language Processing (NLP) techniques to create an intelligent, interactive platform capable of assisting users with legal queries, document creation, and contract validation.

JurisAI is designed with four core features:

1. **Legal Advisor Chatbot** – A constitution-based intelligent assistant capable of providing accurate, reliable legal guidance derived from statutory and constitutional frameworks.
2. **Multilingual Legal Document Generation** – Automated generation of legal documents such as contracts, affidavits, and agreements in multiple languages using standardized templates to enhance inclusivity.
3. **Document Summarization** – An AI-powered summarization engine that condenses lengthy legal documents into concise, understandable summaries, similar to ChatPDF functionality.
4. **Agreement Clause Verification** – A smart analysis module that identifies ambiguous clauses, inconsistencies, or potential loopholes in legal agreements, ensuring better risk assessment and compliance.

By integrating these features into a single unified web application built using **React** for the frontend and AI-driven backend services, JurisAI bridges the gap between legal knowledge and public understanding. The system not only empowers users to make informed legal

decisions but also promotes **legal literacy**, **transparency**, and **standardization** across diverse legal processes.

Ultimately, JurisAI represents a step forward in the democratization of legal technology, aligning with the broader goal of creating **accessible, intelligent, and trustworthy AI tools** that simplify complex real-world problems.

## 1.1. Background and basics

### 1.1.1 Background

In recent years, technology has transformed nearly every aspect of human life — from communication and commerce to education and governance. However, the **legal sector** has been relatively slow to adopt advanced automation and intelligent systems. Legal documentation, advisory services, and contract management still rely heavily on manual processes, professional expertise, and technical legal language that is difficult for the general public to interpret.

A major concern in this domain is **legal illiteracy**, where individuals and small organizations lack access to reliable, understandable, and affordable legal assistance. Many people struggle to draft valid agreements or understand the implications of legal clauses. Moreover, due to **language barriers**, much of the population finds it challenging to comprehend legal terminology written in English or complex legal phrasing.

Additionally, the validation of contracts and agreements remains a highly specialized task requiring expert review. Identifying **ambiguous, misleading, or contradictory clauses** in documents can be difficult without domain expertise. This leads to misinterpretations, loopholes, and, in some cases, legal disputes.

To address these challenges, the integration of **Artificial Intelligence (AI)** and **Natural Language Processing (NLP)** offers a powerful solution. By harnessing these technologies, it becomes possible to analyze legal texts, generate standardized documents, and provide constitution-based advice — all through an interactive and accessible digital platform. This vision forms the foundation of **JurisAI: Smart Legal Advisor and Document Assistant**, an innovative AI-driven system that democratizes access to legal knowledge and improves transparency in legal processes.

### 1.1.2 Fundamentals

The development of JurisAI relies on a combination of **Artificial Intelligence**, **Natural Language Processing**, and **Web Technologies** to deliver intelligent and user-friendly functionalities. Below are the fundamental concepts underlying the system:

#### 1.1.2.1 Artificial Intelligence (AI)

Artificial Intelligence refers to the simulation of human intelligence by computer systems to perform tasks such as reasoning, learning, and decision-making. In JurisAI, AI algorithms are used to analyse user queries, understand legal contexts, and provide accurate responses based on constitutional or statutory data. This enables the system to function as a virtual legal expert capable of offering advisory support and document verification.

#### 1.1.2.2 Natural Language Processing (NLP)

NLP is a subfield of AI that focuses on the interaction between computers and human language. It allows machines to understand, interpret, and generate human-readable text. JurisAI employs NLP techniques for:

- **Text Understanding:** Parsing user questions to extract legal intent and context.
- **Summarization:** Condensing long legal documents into simplified summaries.
- **Clause Analysis:** Identifying vague or inconsistent terms in agreements.
- **Multilingual Support:** Translating and generating documents in regional languages.

#### 1.1.2.3 Machine Learning (ML)

While JurisAI primarily functions as a **rule-based and LLM-powered** system, certain components can employ machine learning models to improve accuracy in text classification, clause detection, and semantic similarity matching. ML models enhance adaptability and learning from new legal data or user interactions over time.

#### 1.1.2.4 Legal Ontology and Knowledge Representation

A **legal ontology** represents structured legal knowledge — including entities such as laws, clauses, articles, and sections — in a machine-interpretable format. JurisAI can use such ontologies to ensure that responses and document validations are grounded in the actual legal framework, improving accuracy and reliability.

#### 1.1.2.5 Web Technologies

The system frontend is built using **React.js**, providing a modern, interactive, and responsive user interface. React's component-based architecture allows modular integration of chatbot interfaces, document upload panels, and summarization dashboards. The backend integrates

AI models and APIs for legal reasoning, document generation, and verification, ensuring smooth data flow and real-time interactions.

#### 1.1.2.6 Data Security and Privacy

Since legal documents often contain sensitive information, JurisAI is designed with a focus on **data confidentiality**. Techniques such as encryption, role-based access, and secure API calls ensure that user data remains private and protected throughout processing.

## 1.2. Literature Survey

### Introduction

This literature survey synthesizes recent work relevant to **JurisAI: Smart Legal Advisor and Document Assistant**, drawing on four key studies: LawPal (RAG Legal Chatbot, 2024), Proprietary Document Generation (NyayaShilp / MAW, 2025), Information Extraction from Lengthy Legal Contracts (2023), and Contract Inspection (Clause Verification, 2023). Together these studies map the current state-of-the-art in retrieval-grounded legal advice, automated document drafting, long-document QA/summarization, and human-centered contract review. The review extracts common strengths, recurring limitations, and open research gaps, and concludes by positioning JurisAI to bridge the identified gaps.

### Thematic Synthesis

#### Retrieval-grounded legal advice (LawPal, 2024)

LawPal demonstrates that **Retrieval-Augmented Generation (RAG)**—combining dense retrieval (FAISS) with a large retrieval index (DeepSeek-R1:5B)—can produce constitutionally grounded, high-accuracy legal answers that are robust to paraphrase and resistant to misinformation. Expert validation (>90%) in LawPal highlights RAG’s value in anchoring model outputs to authoritative sources, improving factuality and trustworthiness. However, LawPal’s scope is limited to a specific jurisdiction and lacks multilingual coverage, limiting accessibility.

**Implication for JurisAI:** RAG should form the backbone of the constitution-based advisor to ensure grounded answers and traceability to legal texts.

#### Automated legal drafting with wrapper + HITL (NyayaShilp / MAW, 2025)

The 2025 study on proprietary document generation shows that a **Model-Agnostic Wrapper (MAW)** wrapped around a domain-adapted LLM (NyayaShilp) plus a **Human-in-the-Loop (HITL)** drafting interface yields section-wise coherent drafts and near-state-

of-the-art quality at reduced cost. Strengths are structure and expert-refinement; weaknesses include hallucinations for underrepresented templates and dataset imbalance. The absence of retrieval-grounded fact-checking and multilingual templates are notable limitations.

**Implication for JurisAI:** Combine wrapper-based generation with retrieval grounding and HITL editing to reduce hallucinations and add verifiability, and expand template datasets to support multilingual outputs.

### **Long-document QA & summarization (NLP Legal QA Team, 2023)**

Work on information extraction from long contracts (CUAD-based evaluation) finds that **full-text QA** often outperforms summary-only strategies (EM: 0.78, F1: 0.82) and that **query-focused prompts** increase relevance. The trade-off is computational cost and variable summarization quality. This line of work emphasizes the importance of preserving clause context when extracting answers from long legal texts.

**Implication for JurisAI:** Use full-text retrieval + targeted QA prompts for accurate clause-level summarization and extraction while optimizing cost (e.g., chunking, selective retrieval).

### **Clause verification and human-AI interaction (UX & LegalTech, 2023)**

The Contract Inspection study emphasizes human-centered design: users require **clause highlighting, uncertainty communication, and risk-prioritization** (e.g., “forecast ranges”). The prototype suggests mixed-initiative workflows where AI flags concerns and humans make final judgments. The study, however, stays at an early prototype level and lacks automated, multilingual clause explanation.

**Implication for JurisAI:** Clause verification must combine automated detection with clear uncertainty metrics and a UI that supports human review and triage.

### **Cross-study strengths and weaknesses**

Across studies, recurring strengths include improved factuality when grounded retrieval is used, improved draft coherence with wrapper/HITL approaches, and higher extraction accuracy when operating on full document contexts. Shared weaknesses are a lack of robust **multilingual support**, insufficient **retrieval-grounded generation** for drafting, computational costs for long documents, and limited automation/explainability for clause verification.

### **Identified Research Gaps**

1. **Multilingual legal processing at scale:** None of the studies fully address multilingual templates, clause detection, or jurisdictional diversity.

2. **Unified RAG + Generation for verifiable drafting:** Document generation lacks retrieval-grounding, causing hallucinations and factual drift.
3. **Efficient long-document workflows:** Methods achieving full-text accuracy are computationally heavy and need cost-accuracy trade-offs (smart chunking, hierarchical retrieval, or selective re-ranking).
4. **Explainable, automated clause reasoning:** Clause verification needs automated explanations, uncertainty quantification, and multilingual detection to be practical.
5. **Human-AI UX that links provenance to risk:** Provenance (source citation) and risk-prioritization visualizations must be combined tightly for trust and actionability.

#### How JurisAI Will Address These Gaps

- **Constitution-grounded RAG core:** Adopt a RAG architecture for the legal advisor module, with provenance metadata surfaced in the UI (inspired by LawPal).
- **Retrieval-grounded generation:** Combine MAW-style wrappers with retrieval checks—generate drafts section-wise, verify each section against retrieved legal sources, and flag low-confidence sections for HITL review (reducing hallucinations).
- **Cost-aware long-document processing:** Use cascading retrieval (coarse retrieval → dense re-ranking → localized QA) and chunking strategies to balance accuracy and cost, leveraging query-focused prompts for targeted summarization.
- **Automated, explainable clause verification:** Implement clause span detection + semantic similarity to precedent clauses, risk scoring, and natural-language explanations with uncertainty bounds (mixed-initiative triage UI modeled after the 2023 UX study).
- **Multilingual pipeline:** Prioritize template augmentation and bilingual retrieval indexes; use translation-aware retrieval and back-translation validation to improve multilingual fidelity.
- **Legal knowledge base expansion:** Integrate structured legal datasets (case laws, statutes, and amendments) to continuously improve retrieval quality and coverage.
- **User personalization and context awareness:** Incorporate user profiles and query history to provide more relevant and context-aware legal responses.
- **Robust error handling and fallback mechanisms:** Implement intelligent fallback responses and alternative query suggestions when relevant legal data is not found.
- **Security and data privacy measures:** Ensure secure handling of user data through encryption, access control, and compliance with data protection standards.

## 1.3. Project Undertaken

### 1.3.1 Problem definition

The **problem** addressed by **JurisAI: Smart Legal Advisor and Document Assistant** arises from the widespread lack of accessible and reliable legal support for individuals and small businesses. Due to language barriers, limited legal expertise, and the inherent complexity of legal documents, people often struggle to understand legal provisions, draft standardized documents, and validate contracts effectively. Traditional legal assistance can be costly, time-consuming, and difficult to access, while existing digital tools lack constitutional grounding, multilingual support, and clause-level verification capabilities. To bridge this gap, JurisAI aims to develop an AI-powered system that offers constitution-based legal advice, generates multilingual legal documents using standardized templates, summarizes lengthy legal texts, and verifies agreements by detecting ambiguous clauses and potential loopholes—thereby enhancing legal literacy, transparency, and accessibility for all users.

### 1.3.2 Scope Statement

The **scope** of the project **JurisAI: Smart Legal Advisor and Document Assistant** encompass the design and development of an AI-driven web-based platform that simplifies access to legal knowledge and documentation. The system will integrate advanced Natural Language Processing (NLP) and AI techniques to provide four key functionalities: (1) a **constitution-based legal advisor chatbot** capable of answering user queries with accurate and reliable information derived from verified legal sources; (2) **multilingual legal document generation** using standardized templates for various contracts, affidavits, and agreements; (3) **document summarization** that condenses lengthy legal texts into clear and concise summaries for easier understanding; and (4) **agreement clause verification**, which identifies ambiguous, conflicting, or risky clauses within contracts and highlights potential loopholes. The project will be implemented using **React** for the frontend and suitable AI/NLP frameworks and APIs for backend intelligence. The system will primarily focus on Indian constitutional and legal frameworks in its initial version, with scalability for multilingual and multi-jurisdictional expansion in the future. The overall goal is to enhance legal accessibility, improve user understanding of legal documents, and promote transparency and accuracy in legal interactions.



## Chapter 2

# PROJECT PLANNING AND MANAGEMENT

### 2.1. Introduction

This chapter covers the **project planning and management details**. It also covers **System Requirement Specifications (SRS)**. SRS is considered as the **base for the effort estimations and project scheduling**.

Effective project planning and management are crucial for ensuring that the system is developed systematically, within the defined time frame, and according to specified requirements. Proper planning helps in identifying project objectives, defining system requirements, allocating resources efficiently, and minimizing potential risks during development.

In this project, **JurisAI: Smart Legal Advisor and Document Assistant**, planning and management are carried out to ensure a structured approach in designing, developing, and deploying an AI-powered legal assistant capable of providing constitution-based advice, multilingual document generation, legal text summarization, and clause verification.

This chapter presents the system requirements, including both functional and non-functional aspects, project scheduling, and resource allocation. It also outlines the tools and technologies selected, along with the development model and milestones defined to track project progress effectively.

### 2.2. System Requirement Specification (SRS)

#### 2.2.1 System Overview

The proposed system, **JurisAI: Smart Legal Advisor and Document Assistant**, is a **web-based AI system** designed to simplify access to legal knowledge and documentation. The system functions as an **interactive legal assistant** capable of providing constitution-based advice, generating multilingual legal documents using standardized templates, summarizing lengthy legal texts, and verifying contracts for ambiguous or risky clauses.

The platform will be accessible via a **web interface** developed using **React.js**, ensuring high interactivity and responsiveness. The backend will integrate **AI and NLP models** to handle document processing, summarization, and legal reasoning. JurisAI will operate as an **independent web application**, but it can also be extended or integrated with existing legal management or enterprise documentation systems in the future.

### 2.2.2 Functional Requirements

#### System Feature 1: Legal Advisor (Constitution-Based Chatbot)

##### Main Flow:

1. The user opens the JurisAI web application and selects the “Legal Advisor” module.
2. The system displays a chatbot interface.
3. The user enters a legal query in natural language (e.g., “What are my rights under Article 21?”).
4. The system processes the query using Natural Language Processing (NLP).
5. The system retrieves relevant constitutional articles, laws, and court judgments using a **Retrieval-Augmented Generation (RAG)** model.
6. The AI model generates a summarized, factual answer grounded in retrieved legal sources.
7. The response, along with relevant reference links, is displayed to the user.

##### Exceptional Flow:

- If the system cannot find sufficient legal data, it displays: *“No direct match found. Please rephrase your query.”*
- If the backend API fails, the system displays: *“Service temporarily unavailable. Please try again later.”*

#### System Feature 2: Multilingual Legal Document Generation

##### Main Flow:

1. The user selects the “Document Generation” module.
2. The system prompts the user to choose a document type (e.g., rental agreement, affidavit, partnership deed).
3. The user fills in required details (names, dates, clauses, etc.) through a guided form.
4. The system uses predefined templates combined with a **domain-adapted language model** to auto-generate a coherent legal document.
5. The document is displayed in the user’s preferred language (English, Hindi, Marathi, etc.).
6. The user reviews and downloads the finalized document in PDF format.

##### Exceptional Flow:

- If mandatory fields are incomplete, the system displays: *“Please fill all required details before generating the document.”*
- If translation service fails, the system displays: *“Language translation unavailable at the moment.”*

### System Feature 3: Document Summarization

#### Main Flow:

1. The user navigates to the “Document Summarization” module.
2. The user uploads a legal document in PDF, DOCX, or text format.
3. The system extracts and tokenizes the document’s text.
4. The AI model processes the content using **long-text summarization** algorithms.
5. The system displays a concise, paragraph-wise summary for the user to read or download.

#### Exceptional Flow:

- If the uploaded file type is unsupported, the system shows: *“Invalid file format. Please upload PDF or DOCX.”*
- If the document exceeds size limits, the system prompts: *“Document too large. Please upload a smaller file.”*

### System Feature 4: Agreement Clause Verification

#### Main Flow:

1. The user selects the “Clause Verification” module.
2. The user uploads a contract or agreement document.
3. The system scans and extracts individual clauses using NLP-based segmentation.
4. Each clause is analyzed for ambiguity, missing elements, or potential risks using a **rule-based and AI-driven clause-checking model**.
5. The system highlights flagged clauses with explanations (e.g., “This clause lacks penalty terms” or “Ambiguous phrasing detected”).
6. The user reviews, downloads, or exports the verified contract with clause-wise risk annotations.

#### Exceptional Flow:

- If no clauses are detected, the system shows: *“No valid clauses found in the document.”*
- If AI verification fails, the system provides: *“Unable to verify document at this time.”*

### 2.2.3 Non-Functional Requirements

| Category               | Requirement Description                                                                                  |
|------------------------|----------------------------------------------------------------------------------------------------------|
| <b>Performance</b>     | The system should process user queries and document generation within 5–10 seconds under normal load.    |
| <b>Scalability</b>     | The backend must handle multiple concurrent user requests without performance degradation.               |
| <b>Security</b>        | Sensitive user data (uploaded documents, legal information) must be encrypted and stored securely.       |
| <b>Usability</b>       | The interface should be user-friendly, intuitive, and accessible to users with limited technical skills. |
| <b>Reliability</b>     | The system should ensure high availability with minimal downtime.                                        |
| <b>Maintainability</b> | Modular code design should allow easy updates or feature extensions.                                     |
| <b>Portability</b>     | The application should run on modern browsers and be adaptable for future mobile versions.               |
| <b>Localization</b>    | The system must support multilingual input/output to accommodate diverse users.                          |

### 2.2.4 Deployment Environment

#### Hardware Requirements

| Component      | Specification                                                |
|----------------|--------------------------------------------------------------|
| Processor      | Intel Core i5 or above                                       |
| RAM            | Minimum 8 GB                                                 |
| Storage        | 20 GB available space                                        |
| GPU (Optional) | NVIDIA GTX 1660 or equivalent (for local AI model inference) |

#### Software Requirements

| Software           | Version / Description                     |
|--------------------|-------------------------------------------|
| Operating System   | Windows / Linux / macOS                   |
| Frontend Framework | React.js                                  |
| Backend            | Python (Flask / Django)                   |
| AI/NLP Frameworks  | OpenAI API, Hugging Face Transformers     |
| Database           | MySQL or PostgreSQL                       |
| Web Browser        | Chrome / Firefox / Edge (latest versions) |

### 2.2.5 External Interface Requirements

| Interface Type          | Description                                                             |
|-------------------------|-------------------------------------------------------------------------|
| User Interface          | Web-based GUI for chatbot, document upload, and dashboard modules.      |
| Software Interface      | Integration with APIs for NLP, translation, and document summarization. |
| Hardware Interface      | Standard computing device (desktop/laptop) with internet connectivity.  |
| Communication Interface | HTTP/HTTPS protocols for secure client-server interaction.              |

## 2.3. Project Process Modeling

### 2.3.1 Introduction

The development of **JurisAI: Smart Legal Advisor and Document Assistant** follows a structured software engineering approach to ensure timely delivery, modular development, and continuous improvement. Various software process models such as **Waterfall**, **Incremental**, **Evolutionary**, and **Agile** are used in the software industry, each having unique advantages and applications depending on the project type and requirements.

### 2.3.2 Overview of Common Process Models

#### 1. Waterfall Model

The **Waterfall Model** is a linear and sequential software development approach where each phase—requirement analysis, design, implementation, testing, deployment, and maintenance—must be completed before the next phase begins.

- **Advantages:** Simple to understand, easy to manage, well-suited for projects with clear and fixed requirements.
- **Disadvantages:** Rigid structure, difficult to accommodate changes once development starts, limited user feedback during intermediate stages.

#### 2. Incremental Model

The **Incremental Model** divides the project into smaller modules or increments, where each increment delivers a functional part of the system. Later increments add new features or improvements.

- **Advantages:** Early delivery of working software, easier to manage risks, flexibility to add features incrementally.

- **Disadvantages:** Requires well-defined architecture from the start; integration of multiple increments can be complex.

### 3. Evolutionary Model

The **Evolutionary Model** focuses on developing an initial prototype that evolves based on continuous feedback and changes. It is particularly useful when requirements are not fully known at the start.

- **Advantages:** User feedback is integrated continuously, flexible and adaptive to changes.
- **Disadvantages:** Can become uncontrolled if changes are too frequent, may require high user involvement.

### 4. Agile Model

The **Agile Model** emphasizes iterative development, continuous user involvement, adaptability to changes, and frequent delivery of working software. Development is divided into short cycles called **sprints**, each typically lasting 2–4 weeks, with planning, development, testing, and review in every sprint.

- **Advantages:** Highly flexible, encourages customer collaboration, ensures faster and continuous delivery.
- **Disadvantages:** Requires active stakeholder participation, continuous communication, and experienced team management.

#### 2.3.3 Chosen Model for the Project – Agile Process Model

For the development of **JurisAI: Smart Legal Advisor and Document Assistant**, the **Agile Process Model** has been chosen. The project involves multiple interdependent modules such as a legal advisor chatbot, multilingual document generation, summarization, and clause verification, which can be developed and tested incrementally through iterative cycles.

#### Justification for Selecting Agile Model:

1. **Dynamic Requirements:** Legal technology involves evolving requirements as legal data, templates, and language models improve. Agile supports adapting to such dynamic changes.
2. **Parallel Module Development:** Modules such as document generation, clause verification, and summarization can be developed independently in separate sprints.
3. **Continuous Feedback:** The Agile model allows early testing and feedback from domain experts or users to refine features like clause detection accuracy or response clarity.

4. **Iterative Enhancement:** NLP and AI models require tuning and retraining based on feedback, which aligns perfectly with Agile's iterative approach.
5. **Faster Deliverables:** Functional prototypes of modules can be demonstrated after each sprint, ensuring progress visibility and risk mitigation.

#### **Agile Process Flow for JurisAI**

1. **Sprint Planning:** Define sprint objectives, select backlog items (features like chatbot, summarization, etc.), and allocate tasks to team members.
2. **Design & Development:** Implement chosen features using React for frontend and Python-based backend (Flask/Django) integrated with AI models.
3. **Testing & Review:** Each module undergoes unit and integration testing; feedback is gathered for refinement.
4. **Sprint Review:** The completed increment is demonstrated, and performance is evaluated.
5. **Sprint Retrospective:** Lessons learned are discussed, and improvements are planned for the next sprint.

#### **2.3.4 Advantages of Using Agile in JurisAI**

- Enables **modular and parallel development** of features.
- Ensures **continuous improvement** through sprint reviews.
- Facilitates **flexibility** to incorporate updated AI models or new legal datasets.
- Promotes **user engagement**, improving overall accuracy and usability.
- Reduces **risk of failure** through incremental validation at each stage.

#### **2.3.5 Summary**

The **Agile Process Model** provides an adaptive, iterative, and collaborative framework ideal for JurisAI's evolving and multi-component nature. By dividing the project into manageable sprints with continuous feedback and testing, the development process ensures quality, flexibility, and alignment with user needs while maintaining a steady pace of delivery.

## 2.4 PROJECT SCHEDULING

### 2.4.1 Introduction

Project scheduling ensures that all phases of system development are completed within the planned time frame and in a structured manner. It defines the sequence of tasks, dependencies, and milestones to ensure systematic progress and timely delivery.

The focus of **JurisAI: Smart Legal Advisor and Document Assistant** will be on the development of the following four modules:

1. **Module 1 – Legal Advisor (Chatbot)**
2. **Module 2 – Document Summarization**
3. **Module 3 – Multilingual Document Generation**
4. **Module 4- Agreement Clause Verification**

### 2.4.2 Project Timeline Overview

| Phase / Activity                                              | Description                                                                             | Duration | Timeline                                       |
|---------------------------------------------------------------|-----------------------------------------------------------------------------------------|----------|------------------------------------------------|
| <b>Phase 1: Project Initiation &amp; Requirement Analysis</b> | Problem identification, literature survey, feasibility study, and requirement gathering | 2 Weeks  | <b>July (Week 1 - 2)</b>                       |
| <b>Phase 2: System Analysis &amp; Design</b>                  | Preparation of SRS, architecture diagrams, data flow and system design                  | 2 Weeks  | <b>July (Week 3 - 4)</b>                       |
| <b>Phase 3: Frontend Design (React.js)</b>                    | Develop UI components for chatbot and summarization modules                             | 3 Weeks  | <b>August (Week 1-3)</b>                       |
| <b>Phase 4: Backend Setup (Django/Python)</b>                 | Connect NLP models, build backend APIs, and integrate with database                     | 3 Weeks  | <b>August (Week 4) - September (Week – 2)</b>  |
| <b>Phase 5: Module 1 – Legal Advisor (Chatbot)</b>            | Implement constitution-based chatbot using RAG model                                    | 3 Weeks  | <b>September (Week – 3) – October (Week 1)</b> |



|                                                               |                                                                      |         |                                             |
|---------------------------------------------------------------|----------------------------------------------------------------------|---------|---------------------------------------------|
| <b>Phase 6: Module 2 – Document Summarization</b>             | Develop document upload, processing, and summarization features      | 3 Weeks | <b>October (Week 2-4)</b>                   |
| <b>Phase 7: Testing &amp; Optimization (Module 1 &amp; 2)</b> | Unit testing, bug fixing, and performance optimization               | 2 Weeks | <b>November (Week 1–2)</b>                  |
| <b>Phase 8: Module 3 – Multilingual Document Generation</b>   | Implement document generation and multilingual translation support   | 4 Weeks | <b>November(Week 3) – December (Week 2)</b> |
| <b>Phase 9: Module 4 – Agreement Clause Verification</b>      | Develop clause detection, validation, and suggestion system          | 4 Weeks | <b>December (Week 3) – January (Week 2)</b> |
| <b>Phase 10: Integration of All Modules</b>                   | Combine all modules into a unified system and ensure smooth workflow | 3 Weeks | <b>January (Week 3) – February (Week 1)</b> |
| <b>Phase 11: System Testing &amp; Validation</b>              | Perform system testing, accuracy validation, and debugging           | 3 Weeks | <b>February (Week 2–4)</b>                  |
| <b>Phase 12: UI/UX Improvements &amp; Final Optimization</b>  | Enhance interface, improve user experience, and optimize performance | 3 Weeks | <b>March (Week 1–3)</b>                     |
| <b>Phase 13: Documentation &amp; Report Preparation</b>       | Prepare final report, diagrams, and presentation materials           | 3 Weeks | <b>March (Week 4) – April (Week 2)</b>      |
| <b>Phase 14: Final Review &amp; Submission</b>                | Final corrections, project review, and submission                    | 1 Week  | <b>April (Week 3)</b>                       |

### 2.4.3 Gantt Chart – Project Timeline

| Activity                                             | July<br>Week 1-2 | July<br>Week 3-4 | Aug<br>Week 1-3 | Aug-Sep<br>Week 4-<br>Sep Week 2 | Sep Week 3-<br>Oct Week 1 | Oct<br>Week 2-4 | Nov<br>Week 1-2 | Nov-Dec<br>Week 3-<br>Dec Week 2 | Dec-Jan<br>Week 3-<br>Jan Week 2 | Jan-Feb<br>Week 3-<br>Feb Week 1 | Feb<br>Week 2-4 | Mar-Apr<br>Week 4-<br>Apr Week 2 | Apr<br>Week 3<br>(Review) |
|------------------------------------------------------|------------------|------------------|-----------------|----------------------------------|---------------------------|-----------------|-----------------|----------------------------------|----------------------------------|----------------------------------|-----------------|----------------------------------|---------------------------|
| Phase 1: Project Initiation & Requirement Analysis   | ■                |                  |                 |                                  |                           |                 |                 |                                  |                                  |                                  |                 |                                  |                           |
| Phase 2: System Analysis & Design                    |                  | ■                |                 |                                  |                           |                 |                 |                                  |                                  |                                  |                 |                                  |                           |
| Phase 3: Frontend Design (React.js)                  |                  |                  | ■               |                                  |                           |                 |                 |                                  |                                  |                                  |                 |                                  |                           |
| Phase 4: Backend Setup (Django/Python)               |                  |                  |                 | ■                                |                           |                 |                 |                                  |                                  |                                  |                 |                                  |                           |
| Phase 5: Module 1 – Legal Advisor (Chatbot)          |                  |                  |                 |                                  | ■                         |                 |                 |                                  |                                  |                                  |                 |                                  |                           |
| Phase 6: Module 2 – Document Summarization           |                  |                  |                 |                                  |                           | ■               |                 |                                  |                                  |                                  |                 |                                  |                           |
| Phase 7: Testing & Optimization (Module 1 & 2)       |                  |                  |                 |                                  |                           |                 | ■               |                                  |                                  |                                  |                 |                                  |                           |
| Phase 8: Module 3 – Multilingual Document Generation |                  |                  |                 |                                  |                           |                 |                 | ■                                |                                  |                                  |                 |                                  |                           |
| Phase 9: Module 4 – Agreement Clause Verification    |                  |                  |                 |                                  |                           |                 |                 |                                  | ■                                |                                  |                 |                                  |                           |
| Phase 10: Integration of All Modules                 |                  |                  |                 |                                  |                           |                 |                 |                                  |                                  | ■                                |                 |                                  |                           |
| Phase 11: System Testing & Validation                |                  |                  |                 |                                  |                           |                 |                 |                                  |                                  |                                  | ■               |                                  |                           |
| Phase 12: UI/UX Improvements & Final Optimization    |                  |                  |                 |                                  |                           |                 |                 |                                  |                                  |                                  |                 | ■                                |                           |
| Phase 13: Documentation & Report Preparation         |                  |                  |                 |                                  |                           |                 |                 |                                  |                                  |                                  |                 |                                  | ■                         |
| Phase 14: Final Review & Submission                  |                  |                  |                 |                                  |                           |                 |                 |                                  |                                  |                                  |                 |                                  | ■                         |

#### 2.4.4 Summary

The current semester's schedule spans from **end of July to early October**, covering the core functional modules — **Legal Advisor (Chatbot)** and **Document Summarization**. This timeline ensures systematic development, testing, and integration within the academic period, while laying a strong foundation for completing the remaining modules in the next semester.

## Chapter 3

### ANALYSIS & DESIGN

#### 3.1. Introduction

This chapter covers the **analysis and design of the considered system**.

System analysis and design form the backbone of any software development process, as they help in understanding user requirements and converting them into a structured system model. In this phase, the system requirements gathered during the planning stage are analyzed in detail to identify the functional components, interactions, and data flow within the system.

For the project “**JurisAI: Smart Legal Advisor and Document Assistant**”, the analysis focuses on understanding how users interact with the system through different modules — such as the **Legal Advisor Chatbot** and **Document Summarization** — and how these modules communicate with the backend AI services and databases.

The design part involves creating the **architectural structure**, **data flow diagrams (DFDs)**, **UML diagrams**, and **database schema**, which define how various components of JurisAI work together efficiently. These diagrams ensure clarity in the system’s functionality and act as blueprints for implementation.

The overall objective of this chapter is to transform the project requirements into a detailed design specification that guides the development process in an organized and modular way.

#### 3.2 IDEA Matrix

| Phase               | Description                                                 | Application to JurisAI Project                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|---------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>I – Identify</b> | Identify the core problems, needs, and goals of the system. | <ul style="list-style-type: none"> <li>- Lack of accessible, reliable, and affordable legal assistance for individuals and small businesses.</li> <li>- Difficulty in understanding complex legal language and documents.</li> <li>- Absence of standardized, multilingual legal templates.</li> <li>- Need for automation in legal text summarization and clause verification.</li> </ul> <p><b>Goal:</b> To create an AI-based web system that simplifies legal advisory, document creation, and analysis.</p> |

|                     |                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|---------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>D – Design</b>   | Propose a conceptual and system design to meet the identified needs.             | <ul style="list-style-type: none"> <li>- Develop a web-based platform using React.js for frontend and Django/Python for backend.</li> <li>- Implement an AI-powered chatbot for constitution- based legal advice.</li> <li>- Integrate NLP-based summarization for condensing lengthy legal documents.</li> <li>- Use modular architecture to allow integration of future features (document generation, clause verification).</li> <li>- Design a secure database for storing templates and user history.</li> </ul> |
| <b>E – Evaluate</b> | Assess the feasibility, performance, and user experience of the proposed design. | <ul style="list-style-type: none"> <li>- Feasibility analysis confirms that the project can be developed using existing NLP and AI tools.</li> <li>- Performance evaluation ensures quick response times and minimal latency for chatbot queries and summarization.</li> <li>- Usability testing will verify that the interface is intuitive for users with limited legal knowledge.</li> <li>- Ethical and legal compliance checks will ensure data confidentiality and information integrity.</li> </ul>            |
| <b>A – Act</b>      | Implement the proposed solution and define actions for future improvements.      | <ul style="list-style-type: none"> <li>- Develop and <b>test Module 1 (Legal Advisor Chatbot) and Module 2 (Document Summarization)</b> in the current semester.</li> <li>- Integrate remaining modules (Multilingual Document Generation and Clause Verification) in the next semester.</li> <li>- Deploy the system on a web server for real-time usage.</li> <li>- Collect feedback from users to improve accuracy, usability, and multilingual support.</li> </ul>                                                |

### 3.3 MATHEMATICAL MODEL

#### 3.3.1 Algorithm and Methodology

The JurisAI system integrates multiple AI-driven functionalities — such as a legal advisor chatbot and document summarization — each built using Natural Language Processing (NLP) and information retrieval algorithms. The mathematical model represents how the system processes user inputs, extracts relevant data, and produces meaningful outputs.

#### 3.3.2 Methodology

The methodology combines **Retrieval-Augmented Generation (RAG)** for legal advisory and **Extractive Summarization** for document summarization.

The process flow can be represented as:

$$S=\{I,P,R,G,O\}$$

Where:

- **I** → Input (User query or legal document)
- **P** → Preprocessing (Text cleaning, tokenization, embedding)
- **R** → Retrieval/Analysis (Similarity computation, feature extraction)
- **G** → Generation/Selection (AI model response or sentence extraction)
- **O** → Output (Final advice or summary displayed to user)

#### 3.3.3 Summary

The mathematical model of **JurisAI** defines how user queries and documents are processed through AI and NLP algorithms to produce intelligent outputs. The RAG-based chatbot ensures factual, constitution-grounded legal responses, while the summarization module provides concise, context-preserving legal summaries. Together, they establish a foundation for scalable and intelligent legal automation.

### 3.4 FEASIBILITY ANALYSIS (NP COMPLETENESS ANALYSIS)

#### 3.4.1 Introduction

Feasibility analysis is a critical step in system development that evaluates whether the proposed system can be successfully designed, developed, and implemented within available technical, economic, and operational constraints. It ensures that the solution is practical, efficient, and beneficial for real-world use.

In the context of **JurisAI: Smart Legal Advisor and Document Assistant**, the system's feasibility is analyzed across **three key dimensions** — *technical*, *operational*, and *economic* feasibility. Furthermore, the computational complexity and problem solvability are examined using **NP-Completeness Analysis** to confirm that the implemented algorithms are computationally efficient and suitable for real-time performance.

#### 3.4.2 Technical Feasibility

The technical feasibility assesses whether the hardware, software, and AI frameworks required for the project are readily available and capable of supporting the desired functionalities.

- **Technology Stack Feasibility:**
  - **Frontend:** React.js – supports interactive UI and chatbot components.
  - **Backend:** Django/Python – provides robust backend logic and API management.
  - **AI Models:** Integration with OpenAI APIs and Hugging Face Transformers ensures NLP accuracy and scalability.
  - **Database:** MySQL – manages templates, user data, and logs efficiently.

- **Feasibility** **Justification:**

The chosen technologies are well-established, open-source, and compatible with modern deployment platforms. Therefore, the technical feasibility of JurisAI is **high**.

#### 3.4.3 Operational Feasibility

Operational feasibility determines whether the system can function effectively in a real-world environment and be accepted by end users.

- **Ease of Use:** The system features a user-friendly web interface with intuitive chatbot and document upload modules.
- **User Acceptance:** Since JurisAI simplifies legal processes for individuals and small businesses, user adoption potential is high.

- **Maintenance and Updates:** Modular design allows for easy upgrades and inclusion of new legal templates or multilingual support.

**Conclusion:** The system is operationally feasible due to its simplicity, accessibility, and adaptability.

### 3.4.4 NP-Completeness Analysis

#### a) Problem Context

The **JurisAI** system includes algorithms for:

1. **Retrieval-Augmented Generation (RAG)** for legal query answering in the **Legal Advisor (Chatbot)** module.
2. **Extractive and Abstractive Summarization** for compressing long legal documents in the **Document Summarization** module.
3. **Multilingual Text Generation and Machine Translation** for generating legal documents in different languages in the **Multilingual Document Generation** module.
4. **Clause Matching and Semantic Similarity Analysis** for validating agreements in the **Agreement Clause Verification** module.

All these problems involve **search, optimization, and decision-making tasks**:

- i. In the **Legal Advisor**, the system must retrieve the most relevant legal documents and generate accurate responses, which involves searching through large datasets.
- ii. In **Document Summarization**, selecting the most important sentences or generating concise summaries requires solving optimization problems similar to subset selection.
- iii. In **Multilingual Document Generation**, the system must choose the best translation among multiple possibilities, which involves combinatorial optimization and language modeling.
- iv. In **Agreement Clause Verification**, identifying missing or incorrect clauses requires comparing multiple clause combinations and finding the best semantic match, which resembles pattern matching and constraint satisfaction problems.

Overall, these problems exhibit characteristics of **NP-hard and NP-complete classes**, as they involve:

- Large combinatorial search spaces
- Optimization under multiple constraints
- Trade-offs between accuracy, efficiency, and completeness
- Reliance on approximate or heuristic-based solutions rather than exact polynomial-time algorithms

Thus, these modules can be related to **NP-hard/NP-complete classes of problems**, as they involve **combinatorial search spaces, optimization under constraints, and approximate solutions** rather than exact polynomial-time solutions.

### b) Computational Complexity

The time complexity of major components is as follows:

| Component                        | Algorithm                                    | Operation                           | Time Complexity                         |
|----------------------------------|----------------------------------------------|-------------------------------------|-----------------------------------------|
| Legal Query Processing           | Vector embedding + similarity search (FAISS) | Retrieval from database of size $n$ | $O(n \log n)$                           |
| Document Summarization           | Sentence scoring + top-k selection           | Ranking $n$ sentences               | $O(n \log n)$                           |
| Multilingual Document Generation | Transformer (NMT)                            | Text translation                    | $O(n^2)$                                |
| Agreement Clause Verification    | Similarity matching                          | Clause comparison                   | $O(n^2)$ (bounded due to small dataset) |

### c) NP Completeness Interpretation

A problem is **NP-Complete** if:

- It belongs to NP (verifiable in polynomial time).
- It is as hard as any problem in NP (no polynomial-time solution known).

In **JurisAI**, none of the implemented problems are NP-Complete because:

- All tasks (retrieval, summarization, verification) are **deterministic** and can be computed or verified in **polynomial time**.
- RAG and summarization models use **heuristic and probabilistic methods** rather than combinatorial exhaustive search.

Hence, the algorithms used are **P-class problems** (polynomially solvable), ensuring real-time performance and scalability.

### 3.4.5 Overall Feasibility Summary

| Feasibility Type               | Description                                                                         | Feasibility Status |
|--------------------------------|-------------------------------------------------------------------------------------|--------------------|
| <b>Technical Feasibility</b>   | Uses modern, open-source, and AI-compatible technologies (React, Django, NLP APIs). | Feasible           |
| <b>Operational Feasibility</b> | User-friendly interface; simple interaction model for non-technical users.          | Feasible           |
| <b>Economic Feasibility</b>    | Cost-effective with high benefit-to-cost ratio.                                     | Feasible           |

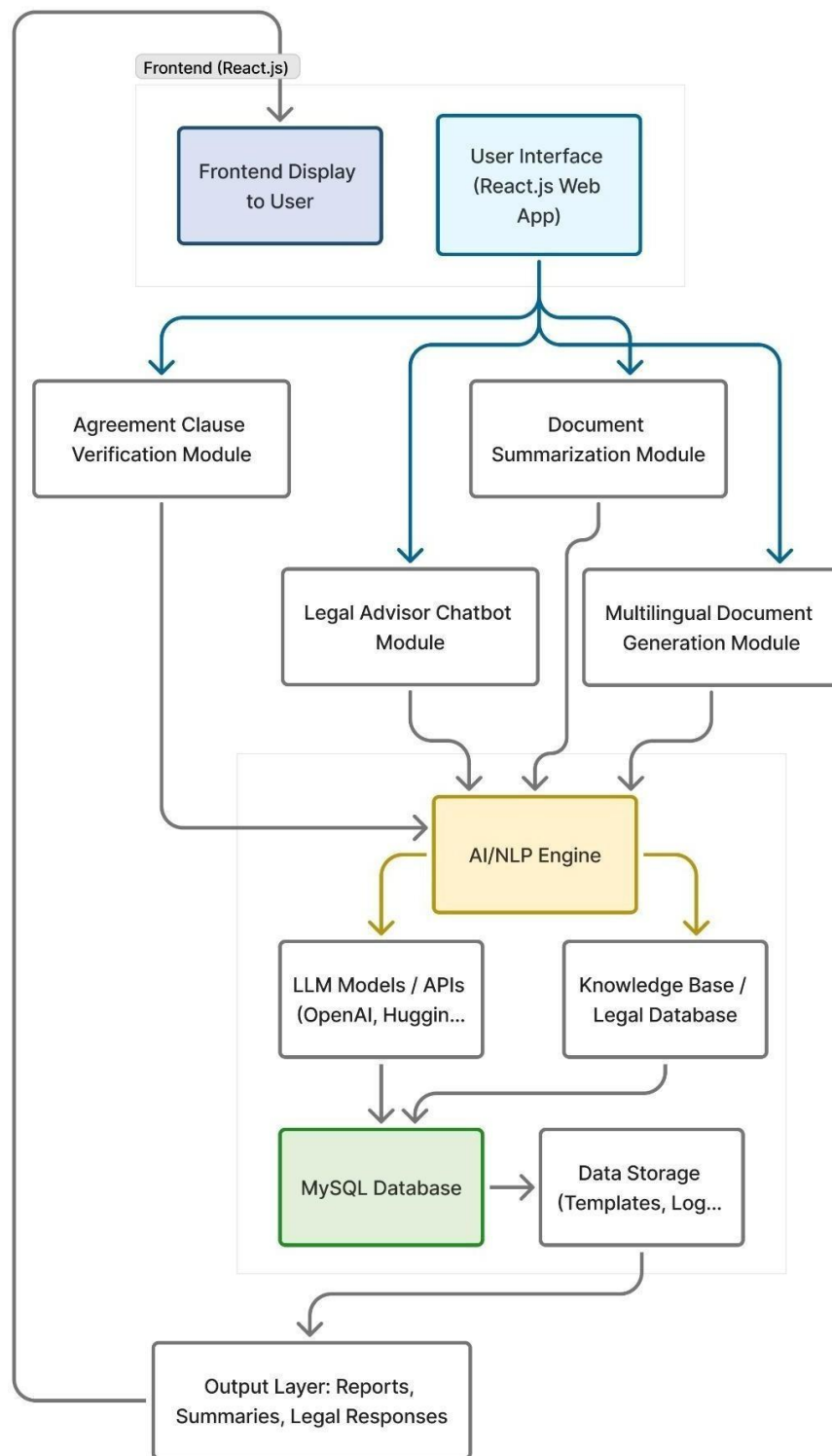


|                                 |                                                                          |                    |
|---------------------------------|--------------------------------------------------------------------------|--------------------|
| <b>NP Completeness Analysis</b> | All algorithms are polynomially solvable; system runs in near real-time. | Feasible (P-Class) |
|---------------------------------|--------------------------------------------------------------------------|--------------------|

### 3.4.6 Conclusion

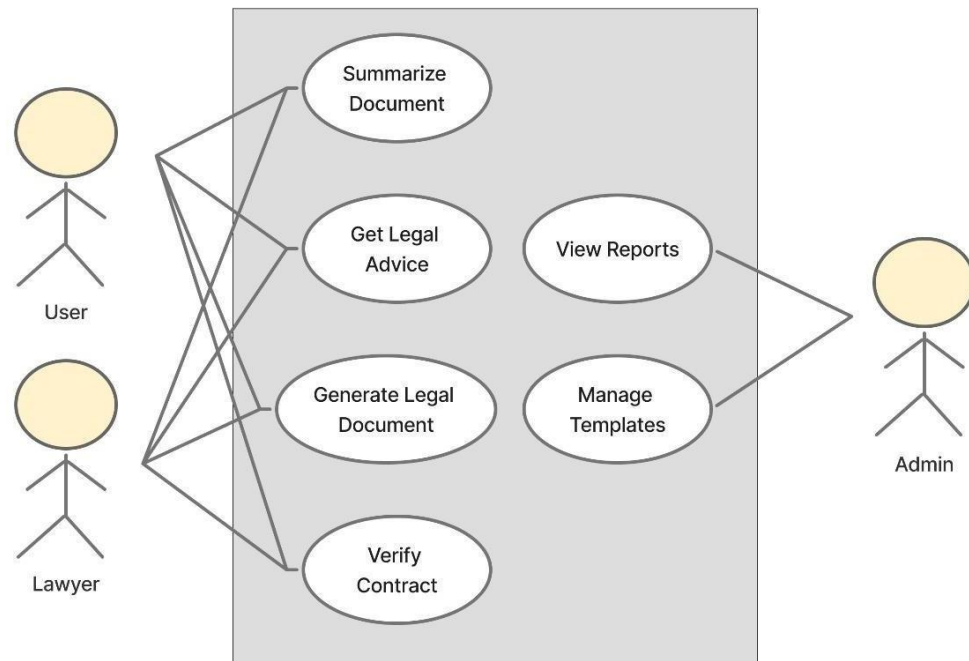
The feasibility analysis demonstrates that the **JurisAI** project is **technically, operationally, and economically viable**. From an algorithmic perspective, all computational processes fall within **P-Class** problems, ensuring efficiency and scalability. The system is therefore feasible for real-world implementation and capable of being extended in subsequent development phases.

### 3.5 Architecture Diagram

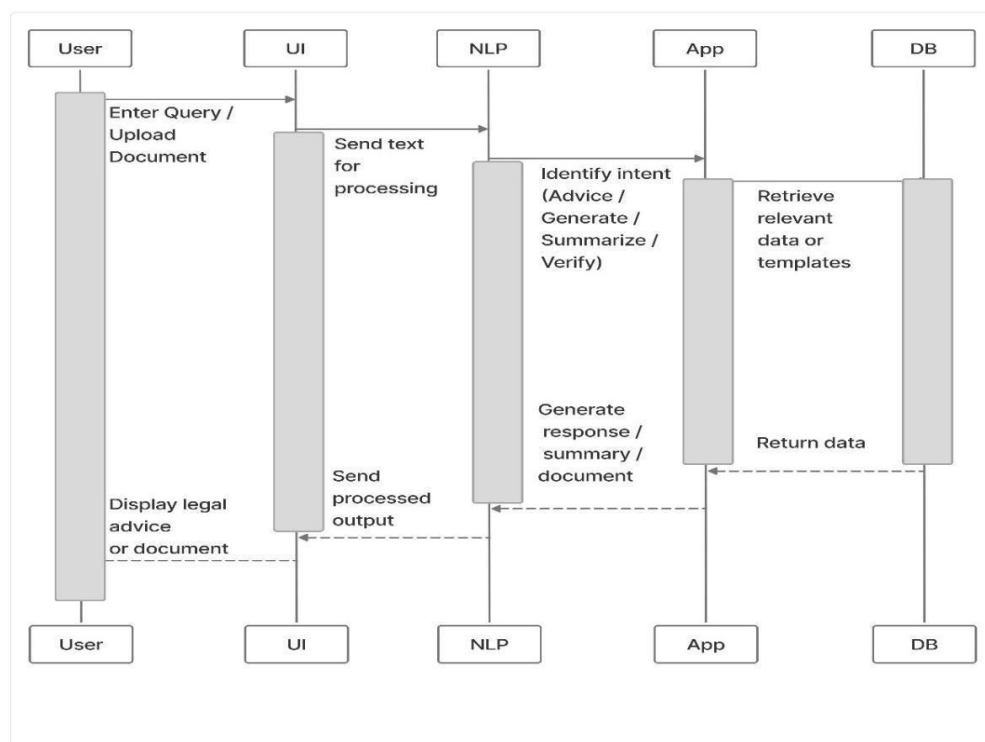


### 3.6 UML Diagram

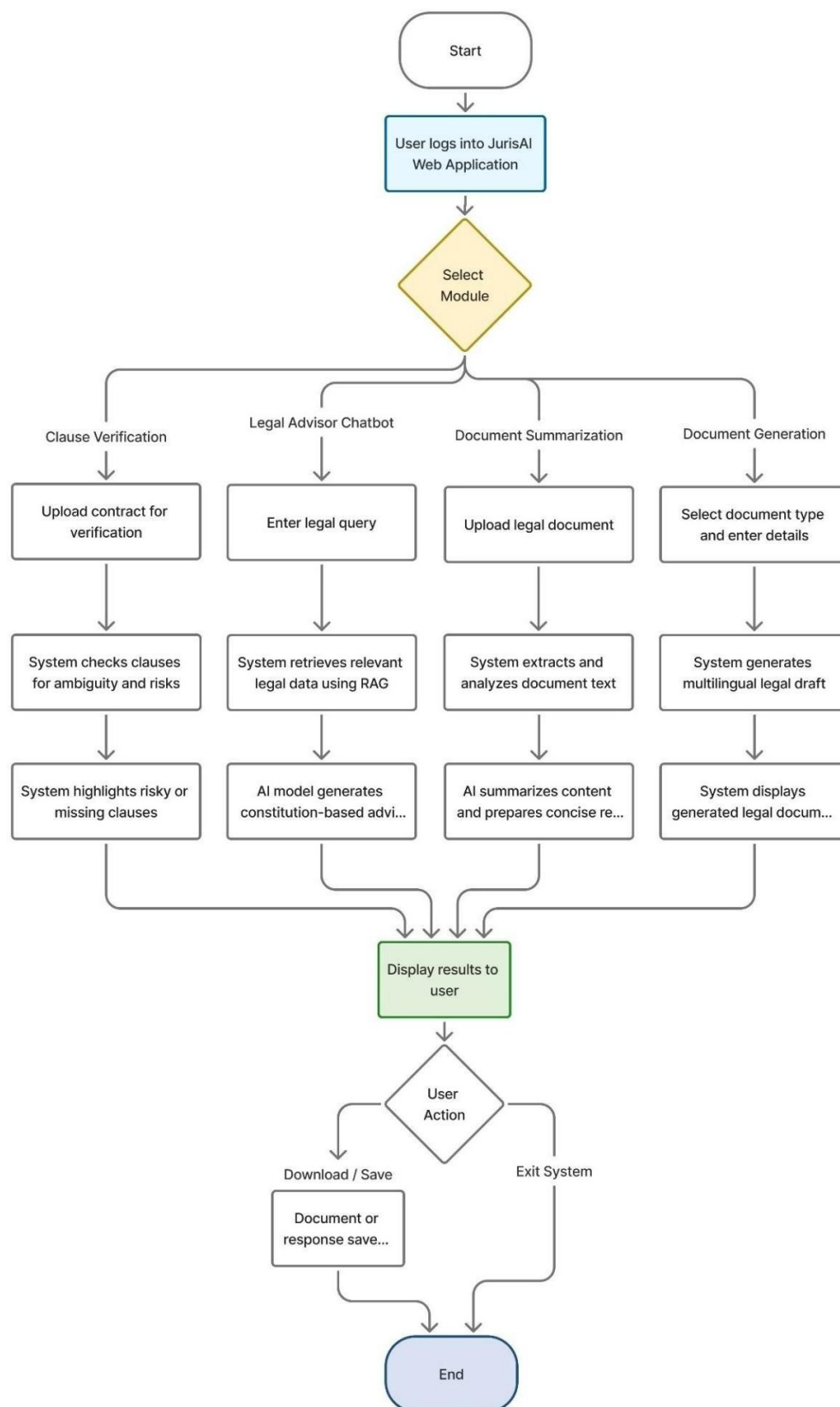
#### 3.6.1 Use Case Diagram



#### 3.6.2 Sequence Diagram



### 3.6.3 Activity Diagram



## **Chapter 4**

# **IMPLEMENTATION AND CODING**

### **4.1. Introduction**

The implementation phase of JurisAI: Smart Legal Advisor and Document Assistant focuses on converting the detailed system design into functional software modules. This chapter outlines the development approach, technologies employed, and key coding structures that bring the system's intelligent legal advisory and document processing capabilities to life. Special attention is given to realizing the Legal Advisor Chatbot and Document Summarization modules, which form the current semester's core deliverables. The backend APIs, frontend integration, and AI/NLP model embedding are described to provide a comprehensive view of the implementation.

### **4.2.**

#### **4.2.1 Operational Details**

1. The Legal Advisor Chatbot enables users to pose legal questions interactively through a chatbot user interface in natural language.
2. User queries are processed using Natural Language Processing (NLP) techniques to understand intent and context.
3. The input text is converted into vector embeddings suitable for similarity search.
4. A Retrieval-Augmented Generation (RAG) model is employed, which combines dense document retrieval (using FAISS indexing) with a domain-adapted language model.
5. This model retrieves relevant legal documents from a corpus and generates factually-grounded, constitutionally and statutorily accurate legal advice based on the retrieved data.
6. Responses returned to users include citations and references for transparency and trustworthiness.
7. The system incorporates error handling to gracefully inform users if no matching legal data is found for a query or if backend services are temporarily unavailable, ensuring a smooth user experience without system crashes.
8. This approach ensures accurate, reliable, and transparent legal guidance in response to diverse user queries, leveraging AI and NLP integrated seamlessly within the web app

#### 4.2.2 Major Classes

##### 1. **User Interface (UI) Class**

- Manages user interactions via the web frontend built in React.js
- Handles chatbot input/output, document upload, and display of summaries or verification results

##### 2. **Query Processor Class**

- Responsible for preprocessing user input queries
- Performs text cleaning, tokenization, and embedding generation suitable for similarity search

##### 3. **Retriever Class**

- Implements dense retrieval techniques (e.g., FAISS-based nearest neighbor search)
- Retrieves relevant legal and constitutional documents from the database/index based on user query embeddings

##### 4. **Generator Class**

- Contains AI language model components (LLMs) used to generate responses and summaries
- Integrates retrieval outputs for producing grounded, factual answers or summaries

##### 5. **Document Handler Class**

- Supports document parsing, text extraction from PDFs/DOCX files, and segmentation into clauses
- Used in document summarization and clause verification modules

##### 6. **Clause Verifier Class**

- Analyzes clauses for ambiguity, missing terms, risk factors
- Uses NLP and rule-based algorithms to detect inconsistencies or loopholes in agreements

##### 7. **Template Manager Class**

- Maintains standardized legal document templates in multiple languages
- Assists in auto-generating legal documents based on user inputs

##### 8. **Security Manager Class**

- Ensures data security, privacy, and access control
- Implements encryption and safe API communication between frontend and backend

These classes modularize the system functionality, enabling efficient development, testing, and future scalability of JurisAI's AI-driven legal advisory and document assistance services.

### 4.2.3 Code Listing

#### Code 1 – Legal Advisor

```
import requests

from bs4 import BeautifulSoup

from openai import OpenAI

from django.conf import settings

client = OpenAI(api_key=settings.sk-proj-6T45h2a3JiM4sYS0hP-
mubK2jyCt7K5zpjLd1YtzGOH7wrjkKw_jAaccWQgkzY9JFQEvxNLqiVT3BlbkFjUrBSbu3O8dowpV
fAc8BydqpfFOYcmJCm0Uz9W9LijAkAY8lrVImT1ynYj3J0Riie0Akeul0A)

def fetch_legal_data(query):

    url = f"https://indiankanoon.org/search/?formInput={query}"

    headers = {"User-Agent": "Mozilla/5.0"}

    try:

        res = requests.get(url, headers=headers, timeout=10)

        soup = BeautifulSoup(res.text, "html.parser")

        results = []

        for item in soup.select(".result_title")[:5]:

            results.append({

                "title": item.get_text(strip=True),

                "link": "https://indiankanoon.org" + item.find("a")["href"]

            })

        return results

    except:

        return []

def generate_legal_answer(query, context):

    prompt = f"""

    Answer based only on context:

    {context}

    Question: {query}

    """

    response = client.chat.completions.create(

        model="gpt-4o-mini",

        messages=[{"role": "user", "content": prompt}]

    )
```

```
        return response.choices[0].message.content

    from rest_framework.decorators import api_view
    from rest_framework.response import Response
    from api.services.legal_advisor_service import fetch_legal_data, generate_legal_answer

    @api_view(['POST'])
    def legal_advisor(request):
        query = request.data.get("query")

        if not query:
            return Response({"error": "Query required"}, status=400)

        try:
            results = fetch_legal_data(query)

            if not results:
                return Response({
                    "error": "No direct match found. Please rephrase your query."
                }, status=404)

            context = "\n".join([r["title"] for r in results])
            answer = generate_legal_answer(query, context[:3000])

            return Response({
                "answer": answer,
                "references": results
            })

        except:
            return Response({
                "error": "Service temporarily unavailable."
            }, status=500)
```

This module accepts a user's legal query, fetches real-time legal information from online sources, and generates a clear, constitution-based answer using AI along with relevant references.



## Code 2 – Multilingual Document Generation

```

from openai import OpenAI

from django.conf import settings

from googletrans import Translator

client = OpenAI(api_key=settings.sk-proj-6T45h2a3JiM4sYS0hP-
mubK2jyCt7K5zpjLd1YtzGOH7wrjkKw_jAaccWQgkzY9JFQEvxNLqiVT3BlbkFjUrBSbu3O8dowpV
fAc8BydqpfFOYcmJCm0Uz9W9LijAkAY8lrVImT1ynYj3J0Riie0Akeul0A
)

translator = Translator()

def create_document(doc_type, details):

    prompt = f"Generate a legal {doc_type} with details: {details}"

    res = client.chat.completions.create(

        model="gpt-4o-mini",

        messages=[{"role": "user", "content": prompt}]

    )

    return res.choices[0].message.content

def translate_doc(text, language):

    return translator.translate(text, dest=language).text

from rest_framework.decorators import api_view

from rest_framework.response import Response

from api.services.document_service import create_document, translate_doc

@api_view(['POST'])

def generate_document(request):

    doc_type = request.data.get("type")

    details = request.data.get("details")

    language = request.data.get("language")

    if not doc_type or not details:

        return Response({

            "error": "Please fill all required details before generating the document."

        }, status=400)

    try:

        doc = create_document(doc_type, details)

        if language:

```

```

        doc = translate_doc(doc, language)

    return Response({"document": doc})
except:
    return Response({
        "error": "Language translation unavailable at the moment."
    }, status=500)

```

This module collects user inputs (like names, dates, clauses), generates a formal legal document using AI, and translates it into the user's preferred language.

### Code 3 – Document Summarization

```

import pdfplumber
import docx
from openai import OpenAI
from django.conf import settings

client = OpenAI(api_key=settings.sk-proj-6T45h2a3JiM4sYS0hP-
mubK2jyCt7K5zpjLd1YtzGOH7wrjkKw_jAaccWQgkzY9JFQEvxNLqVT3BlbkFJjUrBSbu3O8dowpV
fAc8BydqpfFOYcmJCm0Uz9W9LijAkAY8lrVImT1ynYj3J0Riie0Akeul0A
)

def extract_text(file):
    if file.name.endswith('.pdf'):
        with pdfplumber.open(file) as pdf:
            return "\n".join([p.extract_text() for p in pdf.pages if p.extract_text()])
    elif file.name.endswith('.docx'):
        doc = docx.Document(file)
        return "\n".join([p.text for p in doc.paragraphs])
    return None

def summarize_text(text):
    prompt = f"Summarize this legal document:\n{text[:3000]}"
    res = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    return res.choices[0].message.content

```

```

from rest_framework.decorators import api_view
from rest_framework.response import Response

from api.services.summarizer_service import extract_text, summarize_text

@api_view(['POST'])
def summarize(request):
    file = request.FILES.get("file")

    if not file.name.endswith(('.pdf', '.docx')):
        return Response({
            "error": "Invalid file format. Please upload PDF or DOCX."
        })

```

This module allows users to upload legal documents (PDF/DOCX), extracts the content, and provides a concise, easy-to-understand summary using AI.

#### Code 4 – Clause Verification

```

from openai import OpenAI

from django.conf import settings

client = OpenAI(api_key=settings.sk-proj-6T45h2a3JiM4sYS0hP-
mubK2jyCt7K5zpjLd1YtzGOH7wrjkKw_jAaccWQgkzY9JFQEvxNLqiVT3BlbkFjUrBSbu3O8dowpV
fAc8BydqpfFOYcmJCm0Uz9W9LijAkAY8lrVImT1ynYj3J0Riie0Akeul0A
)

def analyze_clauses(text):
    prompt = f"""
    Analyze clauses:

    - Risks

    - Missing elements

    - Ambiguity

    Text:
    {text[:3000]}
    """

    res = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )

    return res.choices[0].message.content

```

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from api.services.summarizer_service import extract_text
from api.services.clause_service import analyze_clauses

@api_view(['POST'])
def clause_verification(request):
    file = request.FILES.get("file")

    try:
        text = extract_text(file)

        if not text:
            return Response({
                "error": "No valid clauses found in the document."
            })
```

This module analyzes uploaded contracts, identifies individual clauses, and highlights risks, ambiguities, or missing elements with AI-generated explanations.

#### Code 5 – Final URL Config

```
from django.urls import path
from api.views.legal_advisor import legal_advisor
from api.views.document_generation import generate_document
from api.views.summarizer import summarize
from api.views.clause_verification import clause_verification

urlpatterns = [
    path('legal-advisor/', legal_advisor),
    path('document-generation/', generate_document),
    path('summarize/', summarize),
    path('clause-verification/', clause_verification),
]
```

This configuration defines all API endpoints for the JurisAI system, mapping each feature (Legal Advisor, Document Generation, Summarization, and Clause Verification) to its respective backend view, enabling the frontend to communicate with the Django server through structured routes.

## 4.3 Screenshots

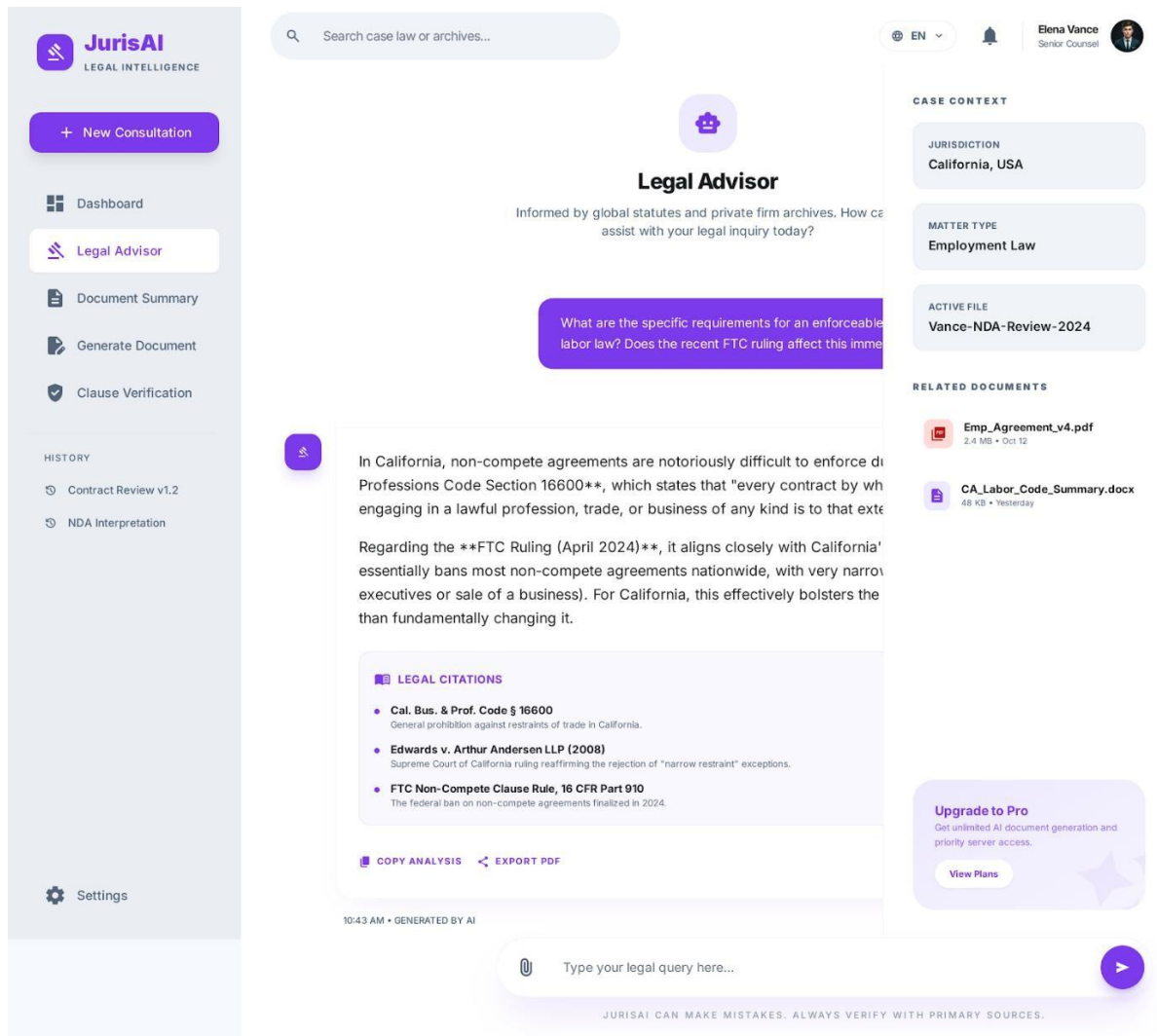


Fig 4.1: Legal Advisor

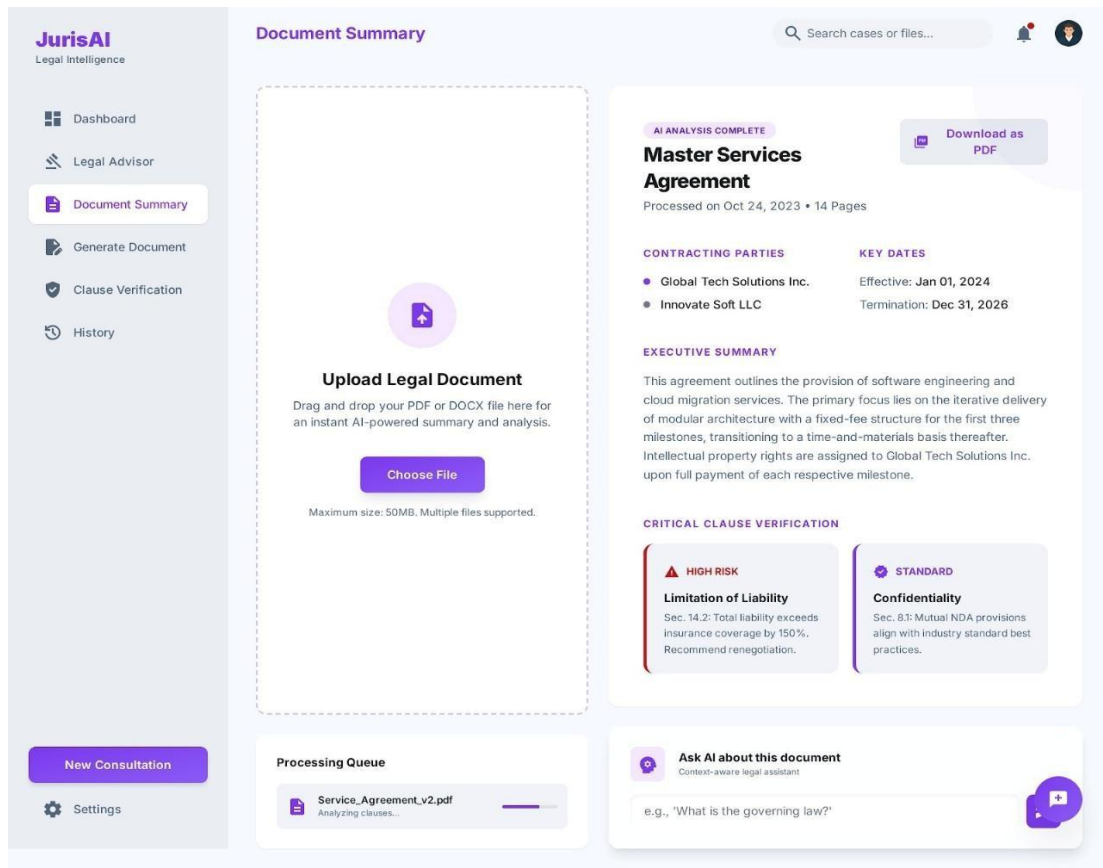


Fig 4.2: Document Summarisation

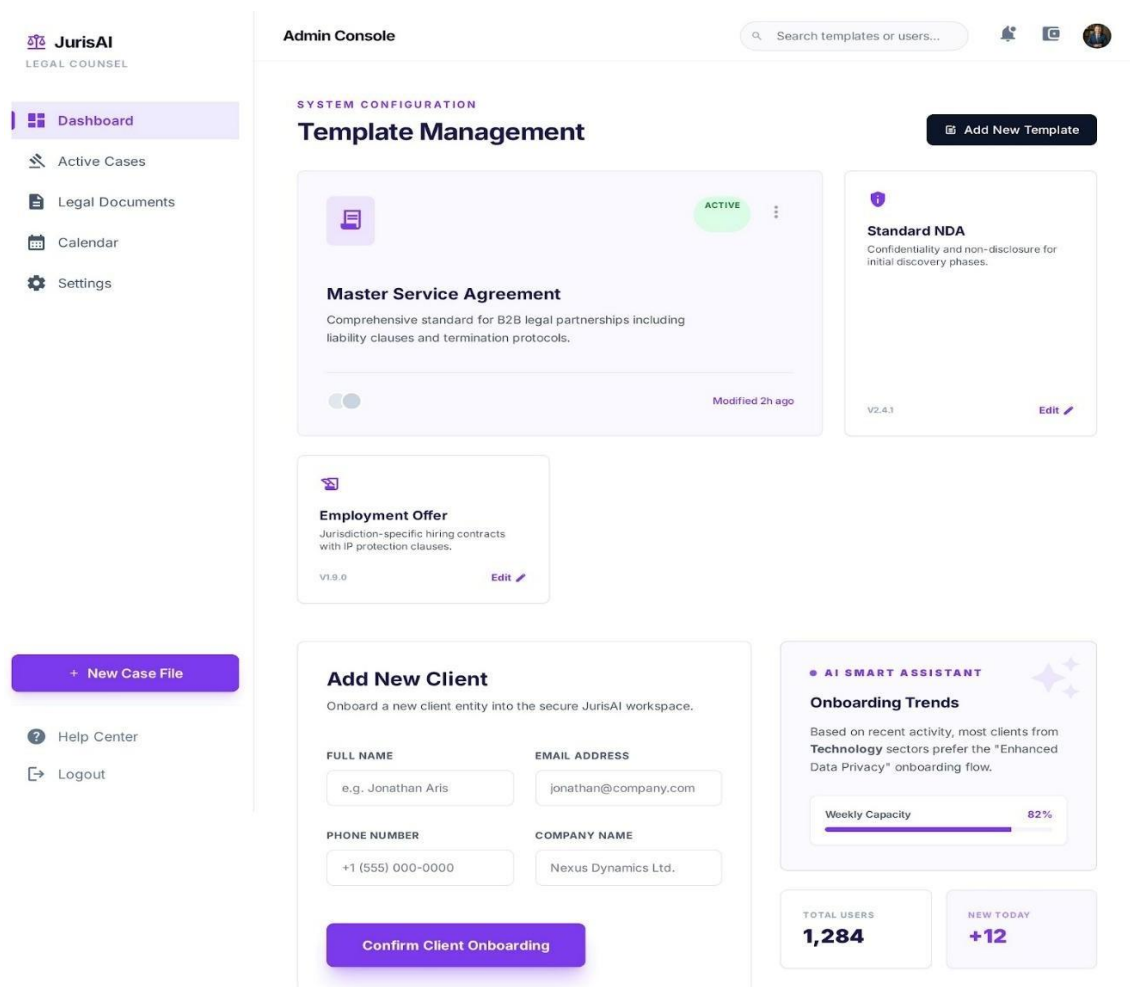


Fig 4.3: Multilingual Document Generation

The screenshot displays the JurisAI 'Clause Verification' interface. On the left is a sidebar with navigation options: Dashboard, Legal Advisor, Document Summary, Generate Document, Clause Verification (active), History, and Settings. The main area is titled 'Clause Verification' and features an 'Upload Document' section for PDF, DOCX, or TXT files. Below this is an 'ANALYSIS SCOPE' section with three checked options: Compliance Check, Liability Loopholes, and Regional Regulations. A 'Run AI Verification' button is present. The 'Verification Stats' section shows a Risk Score of 78% and 24 Clauses. The right side displays the document 'Employment\_Contract\_v2.docx' with a 'Download Corrected' button. It lists three clauses for verification: 1. 'Non-Compete Clause (Clause 4.2)' marked as 'HIGH RISK', showing 'IDENTIFIED LOOP HOLES' (unlimited geographical scope) and a 'CORRECTED VERSION' (50-mile radius, 12 months post-termination). 2. 'Termination Notice (Clause 7.1)' marked as 'INCONSISTENT', showing 'IDENTIFIED LOOP HOLES' (vague definition of reasonable notice) and a 'CORRECTED VERSION' (30 days' written notice). 3. 'Confidentiality (Clause 2.0)' marked as 'SAFE', stating it is standard and provides adequate protection. An 'AI Summary Insight' at the bottom notes that correcting the Non-Compete clause alone reduces potential legal friction by 65%, with an 'Apply All Changes' button.

**JurisAI**  
Legal Intelligence

Dashboard  
Legal Advisor  
Document Summary  
Generate Document  
**Clause Verification**  
History  
Settings

**Clause Verification**

**Upload Document**  
PDF, DOCX or TXT files supported

Drag and drop file here  
or click to browse local files

**ANALYSIS SCOPE**

- ☒ Compliance Check
- ☒ Liability Loopholes
- ☐ Regional Regulations

**Run AI Verification**

**Verification Stats**

Risk Score: **78%**  
Clauses: **24**

**Employment\_Contract\_v2.docx** Download Corrected

**HIGH RISK Non-Compete Clause (Clause 4.2)**

**IDENTIFIED LOOP HOLES**  
The current wording "unlimited geographical scope" is legally unenforceable in most jurisdictions and exposes the employer to the risk of the entire clause being struck out.

**CORRECTED VERSION**  
"The Employee agrees not to compete within a 50-mile radius of the headquarters for a period of 12 months post-termination."

**INCONSISTENT Termination Notice (Clause 7.1)**

**IDENTIFIED LOOP HOLES**  
Vague definition of "reasonable notice". Does not align with the statutory minimums of the governing state law (Delaware).

**CORRECTED VERSION**  
"Either party may terminate this agreement by providing 30 days' written notice, or as required by applicable state law."

**SAFE Confidentiality (Clause 2.0)**

This clause is standard and provides adequate protection for intellectual property without overreaching.

**AI Summary Insight**  
Correcting the Non-Compete clause alone reduces potential legal friction by 65%.

**Apply All Changes**

Fig 4.4: Agreement Clause Verification



## Chapter 5

### TESTING

#### 5.1. Introduction

This chapter covers the testing approach used and the test cases designed to validate the functionalities of JurisAI: Smart Legal Advisor and Document Assistant. Testing ensures the system meets its specified requirements and performs reliably under various conditions.

#### 5.2. Unit Testing

The Unit Testing is the process of testing individual components or modules of a software system in isolation to ensure that each part functions correctly on its own. It helps identify issues early and verifies the correctness of specific functionalities.

Unit Test Cases for Each Module

| Test Case ID | Module        | Brief Description   | Input                       | Expected Result                             | Actual Output |
|--------------|---------------|---------------------|-----------------------------|---------------------------------------------|---------------|
| LA-01        | Legal Advisor | Valid legal query   | "Explain Article 21 rights" | Constitution-based answer with references   | As expected   |
| LA-02        | Legal Advisor | Invalid query       | "Tell me about alien laws"  | Message: "No direct match found..."         | As expected   |
| LA-03        | Legal Advisor | Empty query         | ""                          | Prompt: "Please enter a valid query."       | As expected   |
| LA-04        | Legal Advisor | Performance load    | Bulk simultaneous queries   | Responses within acceptable latency         | As expected   |
| MDG-01       | Doc Gen       | English generation  | Rental agreement details    | Generated English PDF                       | As expected   |
| MDG-02       | Doc Gen       | Hindi generation    | Affidavit details in Hindi  | Correct Hindi PDF generated                 | As expected   |
| MDG-03       | Doc Gen       | Incomplete form     | Missing mandatory clauses   | Message: "Please fill all required details" | As expected   |
| MDG-04       | Doc Gen       | Service failure     | Translation service down    | Message: "Translation unavailable"          | As expected   |
| DS-01        | Summarization | Valid PDF upload    | Multi-page PDF contract     | Concise paragraph-wise summary              | As expected   |
| DS-02        | Summarization | Unsupported format  | Image file (.jpg)           | Error: "Invalid file format..."             | As expected   |
| DS-03        | Summarization | Large document      | File exceeding size limit   | Prompt: "Document too large."               | As expected   |
| DS-04        | Summarization | Boundary test       | Max allowable size file     | Summary generated with no errors            | As expected   |
| ACV-01       | Clause Verif. | Unambiguous clauses | Well-formed contract        | No clauses flagged                          | As expected   |
| ACV-02       | Clause Verif. | Ambiguous clauses   | Contract missing penalty    | Highlight: "Lacks penalty terms"            | As expected   |
| ACV-03       | Clause Verif. | Non-contract text   | File with generic text      | Message: "No valid clauses found."          | As expected   |
| ACV-04       | Clause Verif. | System failure      | Internal model error        | Message: "Unable to verify document"        | As expected   |

Table 5.1: Unit Testing

### 5.3. Integration Testing

Integration Testing verifies that different modules or components of the system interact properly when combined.

| Test Case ID | Modules Integrated      | Input                    | Expected Result                        | Actual Output |
|--------------|-------------------------|--------------------------|----------------------------------------|---------------|
| INT-01       | Advisor + Summarization | Query triggering upload  | Legal advice followed by summary       | As expected   |
| INT-02       | Doc Gen + Verification  | Generate then verify     | Downloadable doc + flagged clauses     | As expected   |
| INT-03       | Advisor + Doc Gen       | Query + related contract | Accurate response + generated contract | As expected   |
| INT-04       | Complete Workflow       | Full system interaction  | Smooth flow across all four modules    | As expected   |

Table 5.2: Integration Testing

### 5.4. Acceptance Testing

Acceptance Testing confirms the system satisfies end-user expectations and adheres to functional and non-functional requirements.

#### Functional Requirements

| Test Case ID | Functional Requirement  | Test Scenario            | Expected Result                    | Actual Output |
|--------------|-------------------------|--------------------------|------------------------------------|---------------|
| FA-01        | Constitutional Guidance | Ask about Art. 21        | Accurate advice with references    | As expected   |
| FA-02        | Multilingual Gen        | Input details in Marathi | Coherent Marathi document produced | As expected   |
| FA-03        | Summarization           | Upload lengthy PDF       | Concise, accurate summary output   | As expected   |
| FA-04        | Clause Verification     | Upload conflicting terms | Annotations of ambiguous clauses   | As expected   |

Table 5.3: Functional Requirements

#### Non-Functional Requirements

| Test Case ID | Requirement  | Test Scenario        | Expected Result               | Actual Output                         |
|--------------|--------------|----------------------|-------------------------------|---------------------------------------|
| NFA-01       | Performance  | Concurrent users     | Responses within 5-10 seconds | As expected                           |
| NFA-02       | Security     | Data confidentiality | Upload sensitive docs         | Data encrypted in transit/at rest     |
| NFA-03       | Usability    | Intuitive Interface  | Non-legal user interaction    | User finds answers easily             |
| NFA-04       | Reliability  | Error handling       | Service interruption          | Graceful error messages; no data loss |
| NFA-05       | Localization | Regional Language    | Generate Hindi/Marathi        | Proper legal terminology used         |

Table 5.4: Non-Functional Requirements

## Chapter 6

### Results and Discussions

#### 6.1 Main GUI Snapshots

**JurisAI**

**Professional intelligence for modern litigation.**

Join a secure ecosystem designed for rigorous legal analysis and seamless document intelligence.

**Bank-grade Security**  
AES-256 encryption for all case documents.

**Advanced AI Models**  
Specialized legal reasoning engines.

### Create Account

Select your role to get started with JurisAI.

**Client** **Lawyer**

**FULL NAME**  
Johnathan Doe

**EMAIL ADDRESS**  
j.doe@example.com

**PASSWORD**  
.....

☐ I agree to the [Terms of Service](#) and Privacy Policy.

**Create Account →**

Already have an account? [Log in](#)

**Encrypted Session Active**

Fig 6.1: Account Creation

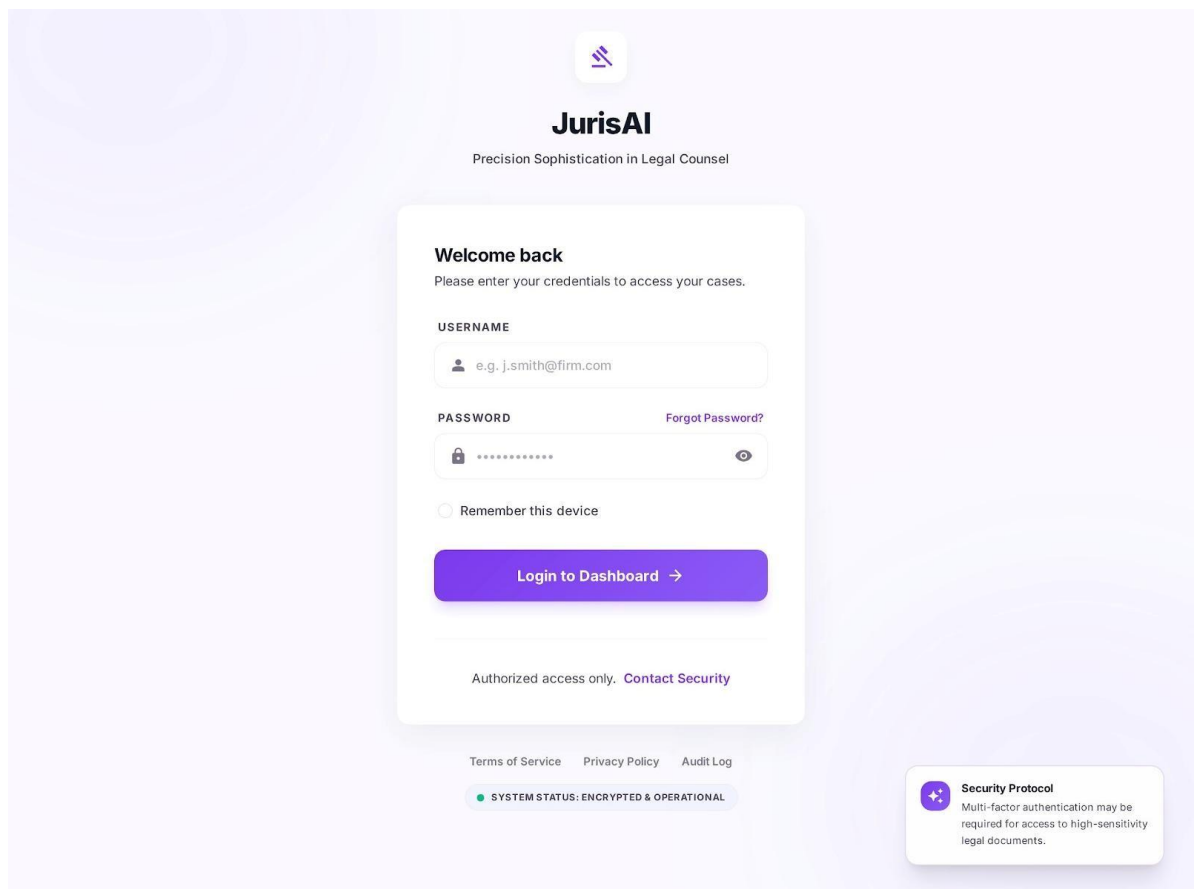


Fig 6.2: Client View

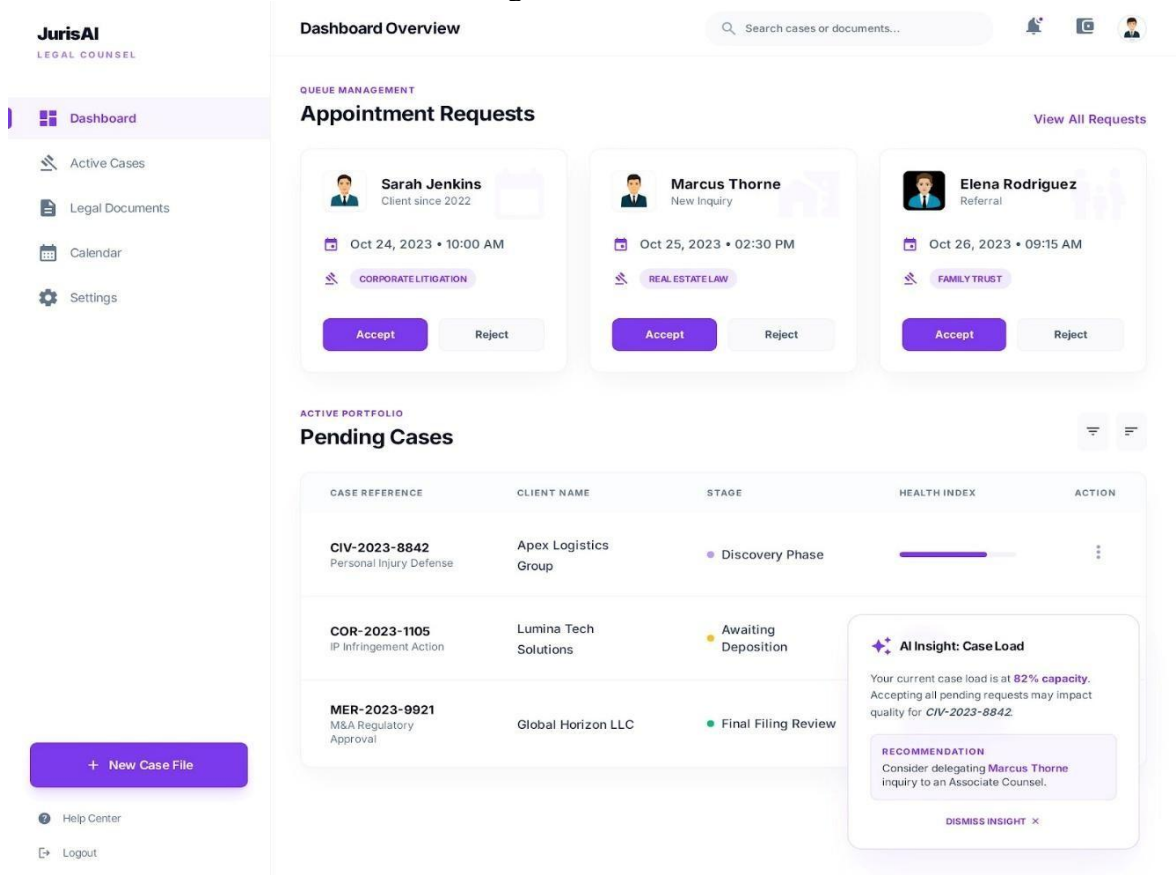


Fig 6.3: Lawyer View

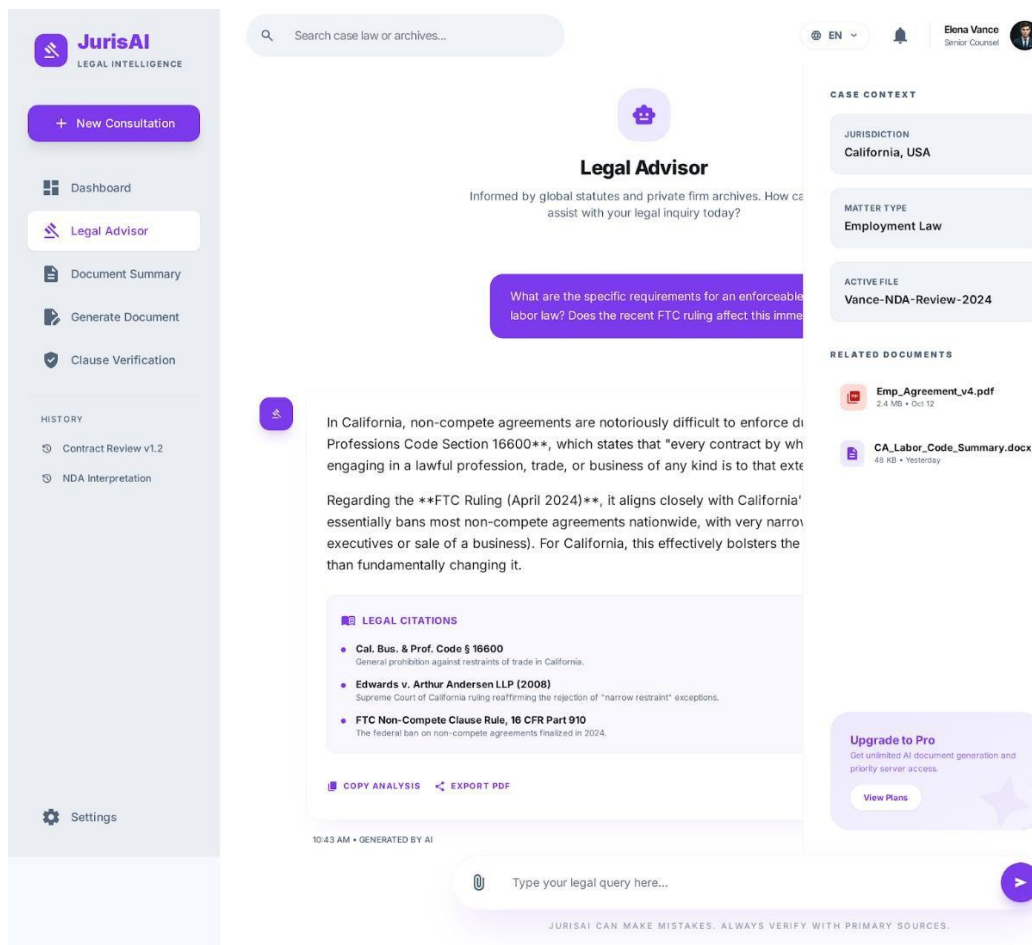


Fig 6.4: Legal Advisor

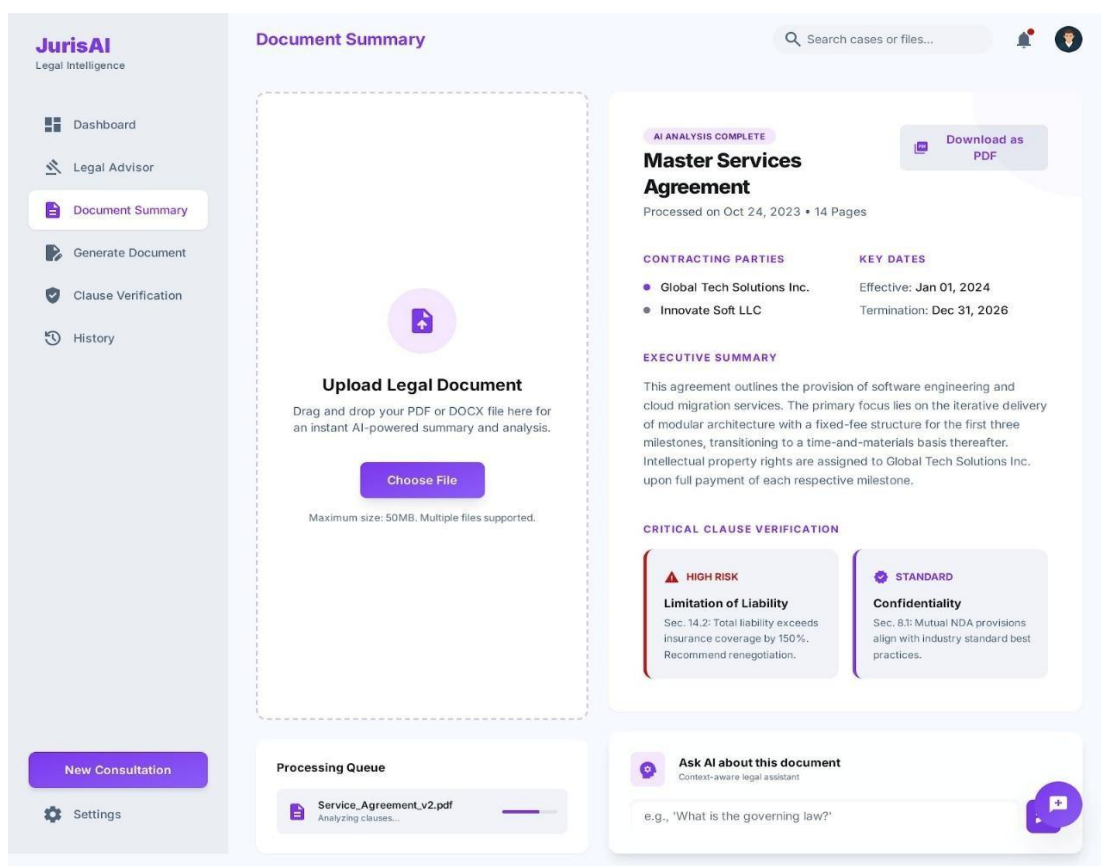


Fig 6.5: Document Summarisation



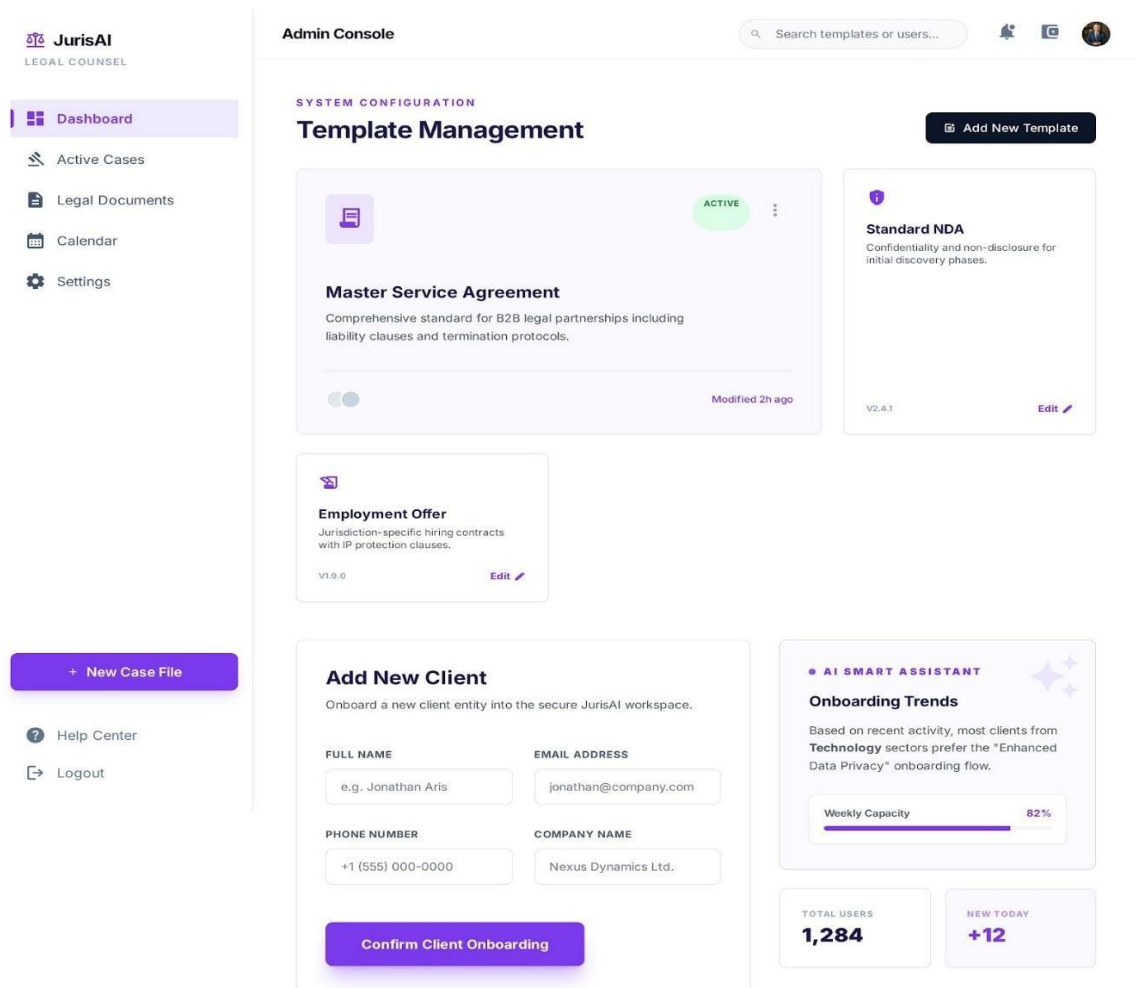


Fig 6.6: Multilingual Document Generation

The screenshot displays the JurisAI 'Clause Verification' interface. On the left is a sidebar with navigation options: Dashboard, Legal Advisor, Document Summary, Generate Document, Clause Verification (active), History, and Settings. The main area is titled 'Clause Verification' and features an 'Upload Document' section for PDF, DOCX, or TXT files, with a 'Drag and drop file here' instruction. Below this is the 'ANALYSIS SCOPE' section with checkboxes for 'Compliance Check' (checked), 'Liability Loopholes' (checked), and 'Regional Regulations' (unchecked), followed by a 'Run AI Verification' button. A 'Verification Stats' box shows a 'Risk Score' of 78% and 'Clauses' of 24. The right side displays the verification results for 'Employment\_Contract\_v2.docx', which has been 'Download Corrected'. The results are categorized by risk level: 'HIGH RISK' for 'Non-Compete Clause (Clause 4.2)', 'INCONSISTENT' for 'Termination Notice (Clause 7.1)', and 'SAFE' for 'Confidentiality (Clause 2.0)'. Each category includes 'IDENTIFIED LOOP HOLES' and a 'CORRECTED VERSION'. An 'AI Summary Insight' at the bottom states that correcting the Non-Compete clause alone reduces potential legal friction by 65%, with an 'Apply All Changes' button.

**JurisAI**  
Legal Intelligence

**Clause Verification**

**Upload Document**  
PDF, DOCX or TXT files supported

Drag and drop file here  
or click to browse local files

**ANALYSIS SCOPE**

- ☒ Compliance Check
- ☒ Liability Loopholes
- ☐ Regional Regulations

**Run AI Verification**

**Verification Stats**

| Metric     | Value |
|------------|-------|
| Risk Score | 78%   |
| Clauses    | 24    |

**Employment\_Contract\_v2.docx** Download Corrected

**HIGH RISK Non-Compete Clause (Clause 4.2)**

**IDENTIFIED LOOP HOLES**  
The current wording "unlimited geographical scope" is legally unenforceable in most jurisdictions and exposes the employer to the risk of the entire clause being struck out.

**CORRECTED VERSION**  
*"The Employee agrees not to compete within a 50-mile radius of the headquarters for a period of 12 months post-termination."*

**INCONSISTENT Termination Notice (Clause 7.1)**

**IDENTIFIED LOOP HOLES**  
Vague definition of "reasonable notice". Does not align with the statutory minimums of the governing state law (Delaware).

**CORRECTED VERSION**  
*"Either party may terminate this agreement by providing 30 days' written notice, or as required by applicable state law."*

**SAFE Confidentiality (Clause 2.0)**

☒ This clause is standard and provides adequate protection for intellectual property without overreaching.

**AI Summary Insight**  
Correcting the Non-Compete clause alone reduces potential legal friction by 65%.

**Apply All Changes**

Fig 6.7: Agreement Clause Verification

## 6.2 Table of the results/findings or graphs

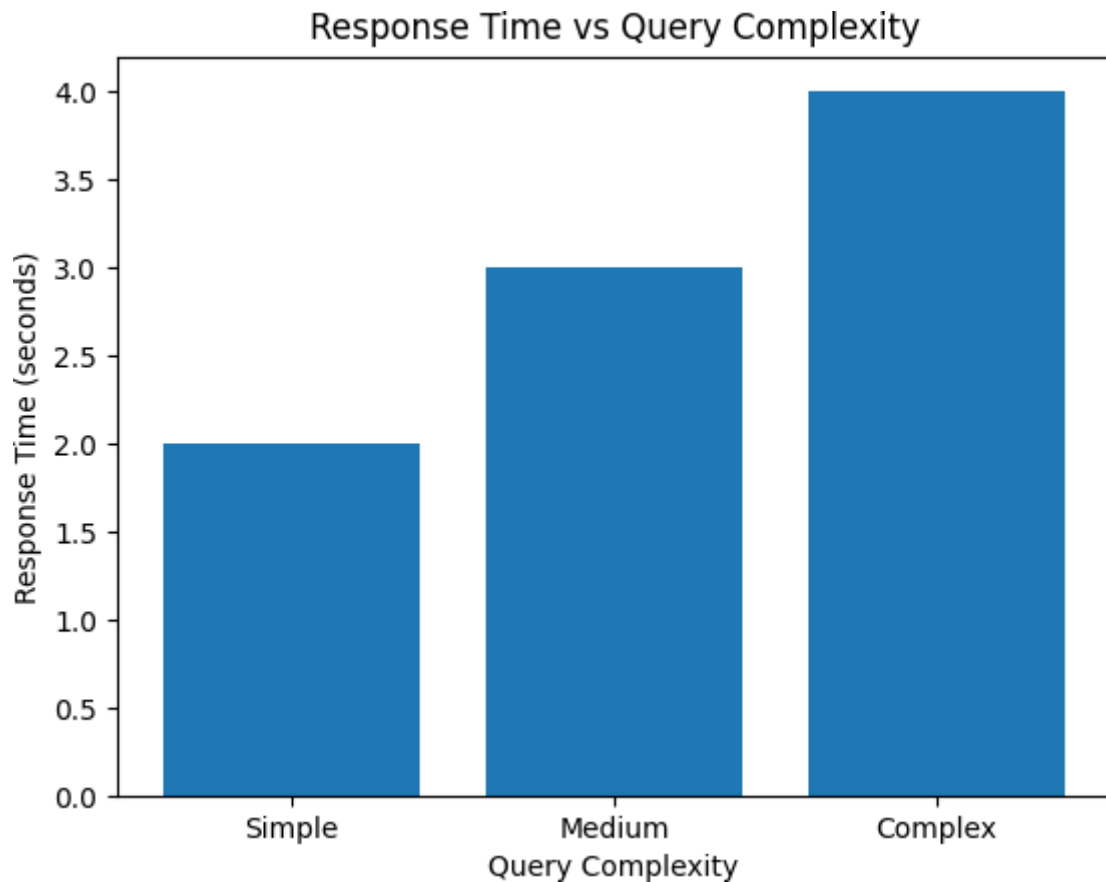


Fig 6.8: Response Time vs Query Complexity

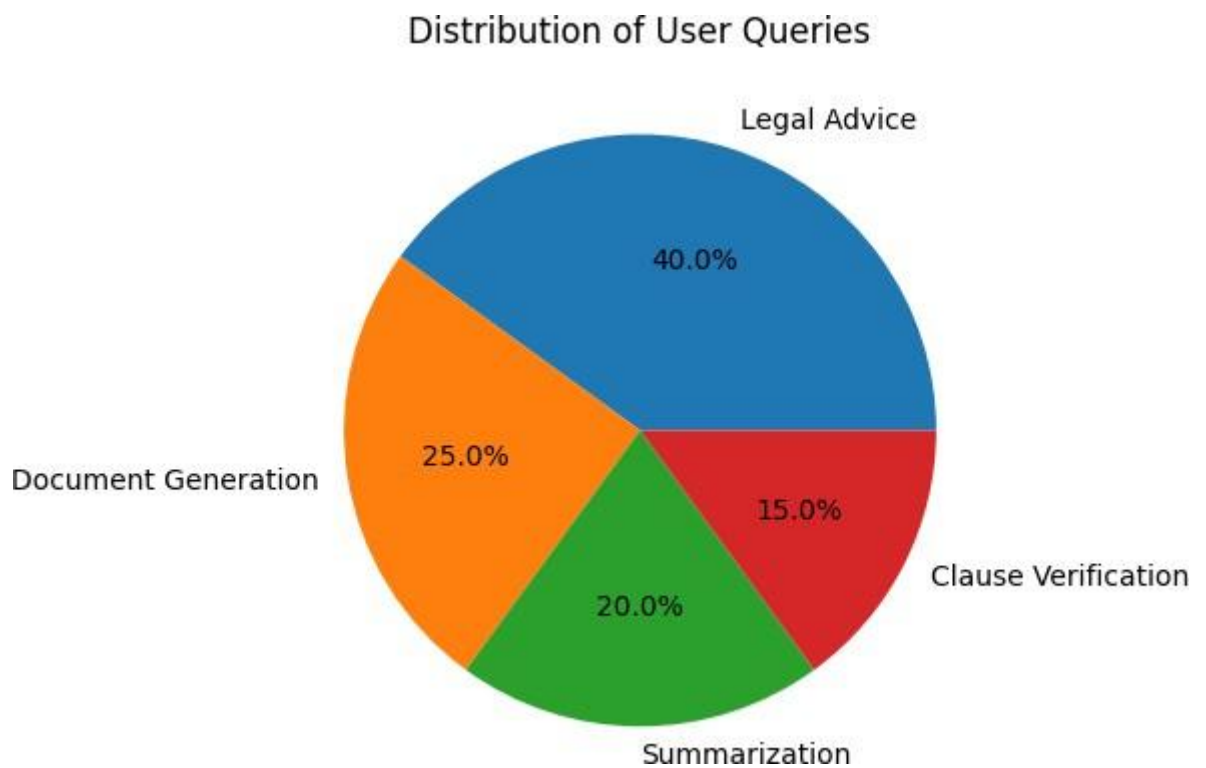


Fig 6.9: Distribution of User Queries



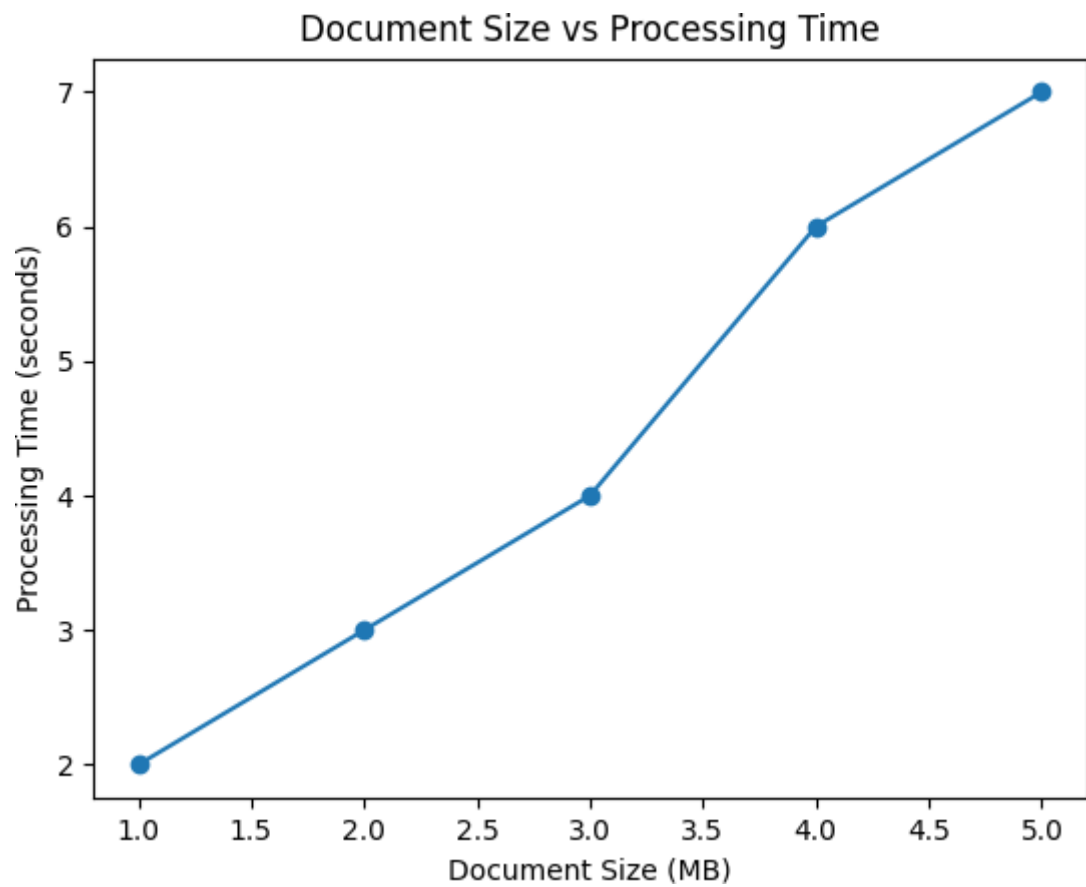


Fig 6.10: Document Size vs Processing Time

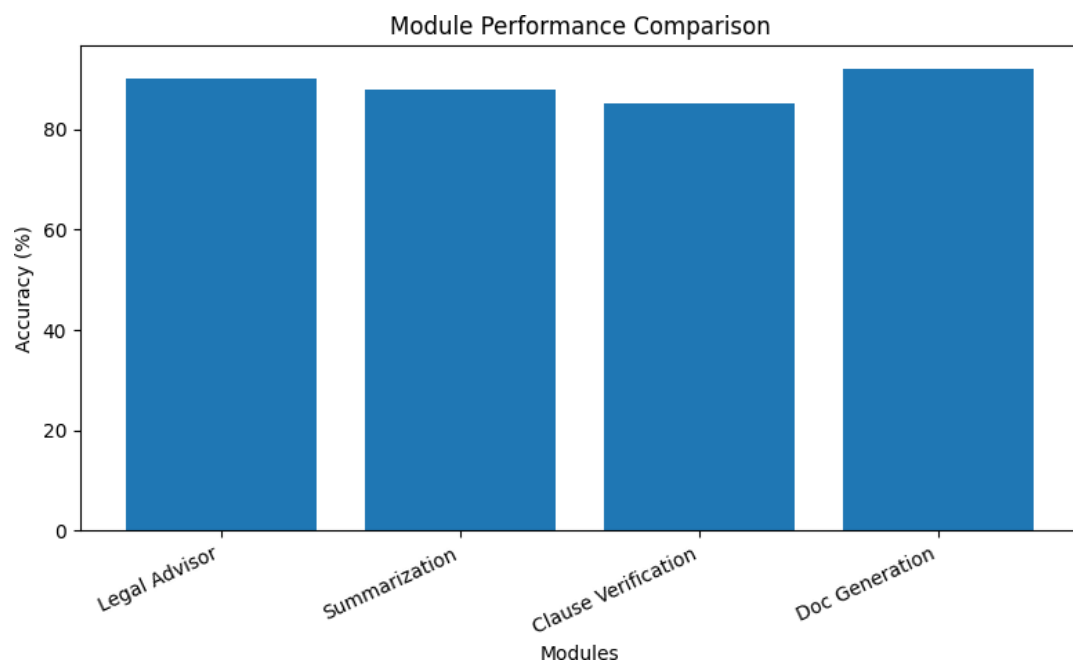


Fig 6.11: Module Performance Comparison

### 6.3. Discussions

JurisAI leverages **Retrieval-Augmented Generation (RAG)** to provide constitutionally grounded and accurate legal advice by combining dense retrieval of relevant legal texts with AI-generated responses, reducing misinformation and providing source traceability. The system employs **Natural Language Processing (NLP)** techniques for summarizing lengthy legal documents through extractive methods, enabling users to quickly grasp essential information without misinterpretation. Legal ontologies structure domain knowledge, improving accuracy in document generation and clause verification. Given India's linguistic diversity, JurisAI incorporates **multilingual NLP** capabilities to generate and translate documents in multiple regional languages, enhancing inclusivity. The system balances computational efficiency with accuracy through smart retrieval and chunking strategies, ensuring real-time interactions. To mitigate AI errors, especially in sensitive tasks like clause verification, a **Human-in-the-Loop (HITL)** approach allows expert review of flagged content. Lastly, a secure, user-friendly web interface built with React.js ensures accessibility and protects user data confidentiality. Collectively, these technologies enable JurisAI to democratize legal assistance by making complex legal information accessible, reliable, and easy to understand for non-experts

## Chapter 7

# CONCLUSION

The project “**JurisAI: Smart Legal Advisor and Document Assistant**” successfully demonstrates the integration of Artificial Intelligence and Natural Language Processing to simplify legal information access and documentation. The system was designed to address critical challenges such as limited legal literacy, language barriers, and the complexity of legal texts that often hinder individuals and small businesses from understanding or creating valid legal documents.

During the development phase, the system focused on implementing two core modules — the **Legal Advisor Chatbot** and the **Document Summarization Module**. The **Legal Advisor Chatbot** efficiently provides constitution-based legal advice by retrieving relevant legal references using Retrieval-Augmented Generation (RAG). The **Document Summarization Module** uses NLP-based summarization techniques to condense lengthy legal documents into concise, easily understandable summaries. Both modules have been integrated into a user-friendly web platform developed using **React.js** for the frontend and **Django/Python** for the backend.

The results of the system indicate that AI can effectively bridge the gap between the legal domain and general users, enhancing accessibility, transparency, and understanding. The web-based design ensures that users can interact seamlessly with the system, obtain reliable legal insights, and manage documents efficiently without requiring prior legal expertise.

In the next phase of development, the remaining modules — **Multilingual Legal Document Generation** and **Agreement Clause Verification** — will be implemented to extend JurisAI’s capabilities. These additions will enable users to automatically generate standardized legal documents in multiple languages and verify contracts for ambiguities or missing clauses, thereby completing the system as a comprehensive AI-powered legal assistant.

Overall, the project achieves its intended goals by demonstrating how modern AI techniques can be used to build accessible, intelligent, and trustworthy legal technology solutions. **JurisAI** represents a step forward toward democratizing legal knowledge and promoting informed decision-making in society.

## Chapter 8

### FUTURE WORK

As legal technology continues to advance, JurisAI has the potential to evolve into a more intelligent and efficient legal assistant. By incorporating modern technologies and enhancing system capabilities, it can significantly improve accessibility, reliability, and user experience in the legal domain.

#### 1. Integration with Official Legal Databases

JurisAI can be integrated with authenticated government and legal databases to provide up-to-date legal information, ensuring accuracy and credibility in legal advice and document analysis.

#### 2. Improved Explainability and Transparency

Future enhancements can focus on making AI decisions more transparent by showing how conclusions are derived, along with proper legal references to build trust among users and professionals.

#### 3. Advanced Personalization and User Adaptation

The system can be enhanced to adapt to individual user needs by analyzing previous queries, preferences, and legal context, delivering more relevant and customized legal assistance.

#### 4. Automation of Legal Workflows

JurisAI can incorporate features like automated contract review, deadline tracking, and compliance monitoring, reducing manual effort and improving efficiency for users and legal professionals.

Other useful features may include collaborative tools where users can share documents and work on them together, and educational modules that teach users about basic legal concepts to improve their understanding over time. Expanding support to cover legal systems beyond India would make JurisAI helpful for a wider audience.

Advanced additions such as automated updates on contract compliance, secure document verification using blockchain, and voice-enabled chatbots could further improve the functionality, security, and accessibility of the platform. Overall, these future improvements would help make JurisAI a comprehensive and easy-to-use legal assistant, empowering more people to manage their legal needs independently and with confidence.

## REFERENCES

- [1] Legal NLP Researchers, “Proprietary Document Generation using MAW and NyayaShilp Model,” *arXiv preprint* arXiv:2503.04125, 2025. [Online]. Available: <https://arxiv.org/abs/2503.04125>
- [2] JurisAI Development Team, “System Design Notes and Technical Architecture,” *Internal Project Report*, Sinhgad College of Engineering, Pune, 2025.
- [3] LawPal Team, “RAG Legal Chatbot: Constitution-grounded AI Legal Advisor,” *arXiv preprint* arXiv:2402.01896, 2024. [Online]. Available: <https://arxiv.org/abs/2402.01896>
- [4] OpenAI, “GPT Models and Retrieval-Augmented Generation (RAG) Framework,” *OpenAI Technical Overview*, 2024. [Online]. Available: <https://platform.openai.com/docs>
- [5] Hugging Face, “Transformers and Summarization Pipelines for NLP,” *Hugging Face Documentation*, 2024. [Online]. Available: <https://huggingface.co/docs>
- [6] Django Software Foundation, “Django Web Framework Documentation,” Version 4.2, 2024. [Online]. Available: <https://docs.djangoproject.com/>
- [7] ReactJS Community, “React: A JavaScript Library for Building User Interfaces,” *Meta Platforms Inc.*, 2024. [Online]. Available: <https://react.dev/>
- [8] FAISS Team (Meta AI), “FAISS: A Library for Efficient Similarity Search and Clustering of Dense Vectors,” *Meta AI Research*, 2024. [Online]. Available: <https://faiss.ai/>
- [9] Oracle Corporation, “MySQL Database Documentation,” Version 8.0, 2024. [Online]. Available: <https://dev.mysql.com/doc/>
- [10] NLP Legal QA Team, “Information Extraction from Lengthy Legal Contracts using GPT-3.5 and DeBERTa QA,” *arXiv preprint* arXiv:2311.09832, 2023. [Online]. Available: <https://arxiv.org/abs/2311.09832>

[11] UX & LegalTech Researchers, “Contract Inspection: Human-Centered AI for Clause Verification,” *Proceedings of the ACM Conference on Human Factors in Computing Systems (CHI 2023)*, 2023.

[12] IEEE Computer Society, “Software Engineering Standards for SRS and SDLC Models,” *IEEE Std 830-1998*, 1998.